The SageLang Programming Language

A Comprehensive Guide

Jacob Yates

July 2026

Contents

The SageLang Programming Language: A Comprehensive Guide **3**
Executive Summary . . . 3
Part 1: Language Overview and Design Philosophy . . . 3
1.1 Design Goals and Target Use Case . . . 3
1.2 Language Characteristics . . . 4
1.3 Execution Model . . . 5
1.4 Performance Characteristics . . . 5
Part 2: Internal Architecture and Core Modules . . . 6
2.1 Module Dependency Graph . . . 6
2.2 Lexer (lexer.c / lexer.h) . . . 6
2.3 Parser (parser.c) . . . 7
2.4 Value System (value.h / value.c) . . . 7
2.5 Environment and Scoping (env.h / env.c) . . . 9
2.6 Garbage Collector (gc.h / gc.c) . . . 9
2.7 Interpreter (interpreter.c / interpreter.h) . . . 10
2.8 Module System (module.h / module.c) . . . 13
Part 3: Language Features by Phase . . . 14
Phase 1–3: Core Language (Arithmetic, Variables, Control Flow) . . . 14
Phase 4: Garbage Collection . . . 15
Phase 5–6: Data Structures and OOP . . . 15
Phase 7: Advanced Control Flow, Exceptions, Generators . . . 16
Phase 8: Modules . . . 17
Phase 9: Low-Level Programming . . . 17
Part 4: Writing SageLang Programs . . . 20
4.1 Basic Syntax Patterns . . . 20
4.2 Control Flow Patterns . . . 21
4.3 Functions and Closures . . . 22
4.4 Structs, Enums, Traits, and Classes . . . 23
4.5 Exception Handling . . . 25
4.6 Generators . . . 25
4.7 Modules . . . 25
4.8 Defer Statements . . . 26
4.9 Pattern Matching (match/case/default) . . . 27
Part 5: Implementation Details and Runtime Behavior . . . 27
5.1 Memory Layout and Object Representation . . . 27
5.2 Call Stack and Environments . . . 28
5.3 Exception Propagation . . . 28
5.4 Generator State Management . . . 28

5.5 Operator Precedence and Associativity . . . 28
5.6 Type Coercion and Truthiness . . . 29
Part 6: Compilation and Execution . . . 29
6.1 Building the Interpreter . . . 29
6.2 Main Entry Point (main.c) . . . 31
6.3 Program Execution Flow . . . 33
6.4 Runtime Backends . . . 33
Part 7: Advanced Topics and Internals . . . 34
7.1 Memory Leaks and GC Limitations . . . 34
7.2 Performance Characteristics . . . 35
3 Current Recipe Benchmark . . . 35
3 Sage vs Python 3 Benchmarks . . . 36
7.3 Extending SageLang with Native Functions . . . 36
7.4 Debugging and Introspection . . . 37
7.5 Interpreter Safety Limits . . . 37
Part 8: Future Directions and Design Notes . . . 38
8.1 Planned Features (Incomplete Implementations) . . . 38
8.2 Design Decisions and Rationale . . . 38
Part 9: Bundled Library Modules (lib/) . . . 38
9.1 Arrays Module (`lib/arrays.sage`) . . . 38
9.2 Strings Module (`lib/strings.sage`) . . . 39
9.3 Dicts Module (`lib/dicts.sage`) . . . 40
9.4 Iter Module (`lib/iter.sage`) . . . 40
9.5 Stats Module (`lib/stats.sage`) . . . 41
9.6 Utils Module (`lib/utils.sage`) . . . 41
9.7 Assert Module (`lib/assert.sage`) . . . 42
9.8 Math Module (`lib/math.sage`) . . . 42
9.9 GPU Libraries . . . 43
9.10 OS Development Libraries . . . 44
9.11 Networking Libraries . . . 47
9.12 Cryptography Libraries . . . 47
9.13 Machine Learning Libraries . . . 47
9.14 CUDA Libraries . . . 48
9.15 Standard Library (`lib/std/`) . . . 49
9.16 LLM / Neural Network Libraries . . . 50
9.17 Agent Framework (`lib/agent/`) . . . 51
9.18 Chatbot Framework (`lib/chat/`) . . . 52
9.19 UI Widget Library (`lib/graphics/ui.sage`) . . . 52
9.20 AVR Assembler & Arduino Uno Support (`core/boards/AVR/`) . . . 53
Part 13: Self-Hosting (Phase 13) . . . 54
13.1 Architecture . . . 54
13.2 Running Self-Hosted Code . . . 54
13.3 Key Design Decisions . . . 54
13.4 Feature Coverage . . . 55
13.5 Test Suite . . . 55
Part 10: Native Standard Library Modules (Phase 11) . . . 55
10.1 Math Module . . . 56
10.2 IO Module . . . 56
10.3 String Module . . . 56
10.4 Sys Module . . . 57
10.5 FAT Module . . . 57
Part 11: Concurrency and Async/Await (Phase 11) . . . 57
11.1 Thread Module . . . 57
11.2 Async/Await . . . 58

11.3 GC Thread Safety . . . 59
Part 12: Developer Tooling (Phase 12) . . . 59
12.1 REPL (Read-Eval-Print Loop) . . . 59
12.2 Code Formatter . . . 60
12.3 Linter . . . 60
12.4 Syntax Highlighting . . . 60
12.5 Language Server Protocol (LSP) . . . 61
Example Programs . . . 61
Part 14: Networking (Phase 14) . . . 62
14.1 Socket Module . . . 62
14.2 TCP Module . . . 62
14.3 HTTP Module . . . 63
14.4 SSL Module . . . 63
14.5 Important Notes . . . 64
Part 15: JSON Library (cJSON Port) . . . 64
15.1 Basic Usage . . . 64
15.2 Creating JSON . . . 64
15.3 Sage-Native Conversion . . . 65
15.4 API Reference . . . 65
15.5 Important Notes . . . 66
Conclusion . . . 66
GPU Graphics (Vulkan + OpenGL) . . . 66
Build Requirements . . . 66
Quick Start: Empty Window . . . 67
Quick Start: Triangle . . . 67
GPU Module API Reference . . . 68
Key Constants . . . 70
Demos . . . 71
9.21 Hardware Natives (Embedded Targets) . . . 71
Appendix: Quick Reference . . . 72
Keywords . . . 72
Built-in Functions . . . 73
Operators . . . 73

The SageLang Programming Language: A Comprehensive Guide

Executive Summary

SageLang is a **Python-inspired, systems-oriented programming language** written in C. It combines familiar Python syntax (indentation-based blocks, dynamic typing) with low-level systems capabilities (garbage collection, exception handling, generators, and module imports). The language now supports ten execution backends (C, LLVM IR, native assembly, bytecode VM, SageMetal VM, JIT, AOT, Kotlin/Android) and a self-hosted interpreter written in Sage itself. As of v3, Sage features structural value equality in uniqueness checks, safe non-hanging string/value repeating, and robust tab/whitespace token checks in sandbox security guards. This guide documents the language design, internal architecture, runtime semantics, and practical usage patterns derived from the complete C source implementation.

Part 1: Language Overview and Design Philosophy

1.1 Design Goals and Target Use Case

SageLang is designed as an **educational and practical embedded scripting language** that:

- **Bridges Python and C**: Provides Python-like syntax and dynamic typing while running in a minimal C footprint suitable for the RP2040.
- **Supports Systems Programming**: Unlike pure Python, SageLang includes explicit garbage collection control, class-based OOP, and low-level value representation suitable for embedded contexts.
- **Phases Incrementally**: The language grows through discrete, completed phases (1–18), each adding a cohesive feature set without breaking prior functionality.
- **Prioritizes Correctness**: Uses explicit memory management (via mark-and-sweep GC), scoped environments, and exception handling rather than relying on OS-level resource management.

1.2 Language Characteristics

Feature	Details
Syntax	Python-like: indentation-based blocks, `:` terminators, keywords like `let`, `var`, `proc`, `class`
Type System	Dynamically typed; values carry runtime type tags (numbers, strings, bools, arrays, dicts, tuples, classes, generators)
Scoping	Lexical scoping via nested environments; each block/function/class creates a child environment
Memory	Mark-and-sweep (tracing), ARC (reference counting), and ORC (optimized reference counting with cycle detection) modes
OOP	Class-based inheritance with single parent, methods, instance fields, `self` parameter, `super.init()` auto-self
Control Flow	`if/else`, `while`, `for...in`, `break`, `continue`, `return`, `try/catch/finally`, `raise`, `yield`, `defer`
Data Structures	Arrays (dynamic), dicts (string-keyed), tuples (immutable), slicing, indexing
Functions	First-class `proc` declarations with closures; anonymous `proc` expressions (`proc(x): body end`); native C functions
Generators	Full `yield` support with resumable state; `next()` function to iterate
Exceptions	`try/catch/finally/raise` with explicit exception objects and message strings
Modules	`import module`, `import module as alias`, `from module import x, y`, `from module import x as y`. The same module can be imported under multiple bindings.
Standard Library	Native modules: `math`, `io`, `string`, `sys`, `thread`, `fat`, `socket`, `tcp`, `http`, `ssl`
Networking	POSIX sockets, TCP, HTTP/HTTPS (libcurl), SSL/TLS (OpenSSL)
Concurrency	Multi-threaded `proc`, `async/await`, atomics, semaphores, condvars, rwlocks, SMP detection
GPU Graphics	Vulkan + OpenGL 4.5 backends, handle-based API, 100+ functions, Android support
UI Widgets	Immediate-mode GUI: windows, panels, buttons, sliders, menus, text inputs

Feature	Details
Compilation	C backend, LLVM IR (with GPU support), native assembly (x86-64, aarch64, rv64, mips), JIT self-extracting executables (with module bundling), plus initial profile suffixes for bare-metal / OSdev / UEFI targets

LLVM codegen has an additional import optimization path: both the C LLVM backend and the self-hosted LLVM backend resolve `from module import CONST` for foldable top-level constants during code generation (including `as` aliases).

1.3 Execution Model

SageLang uses a shared front-end with multiple execution backends:

1. **Source Code** → **Lexer** (tokenization with indentation tracking)
2. **Tokens** → **Parser** (recursive descent, builds AST)
3. **AST** → one of five backends:
   - **AST interpreter** (tree-walking, default)
   - **Bytecode compiler + VM** (stack-based, faster for hot loops)
   - **SGVM binary** (`--sgvm`, binary bytecode artifact)
   - **Lily Transpiler** (`--compile-to-lily`, `--compile-from-lily`)
   - **C codegen** (`--emit-c` / `--compile`)
   - **LLVM IR** (`--emit-llvm` / `--compile-llvm`, with GPU support)
   - **Native assembly** (`--emit-asm` / `--compile-native`, x86-64/aarch64/rv64/mips)
   - **Freestanding ELF** (`--compile-bare`, bare-metal kernel output)
   - **UEFI PE** (`--compile-uefi`, EFI application output)
   - **SageMetal VM** (`make metal-vm`, freestanding bytecode object)
4. **Runtime Execution**: Execution by the `MetalVM` engine integrated into `sage` or via standalone `sgvm` tool.
5. **Runtime Values** stored in the shared **heap** managed by **GC**

All execution modes share the same object model: a **global environment**, nested **child environments** for scopes, and **tagged `Value` objects** that are either immediate (numbers, bools) or GC-managed heap values (arrays, dicts, strings, classes, instances, functions, generators).

For LLVM-compiled workloads, `from module import CONST` is resolved at compile time for foldable top-level module constants (numbers/strings/bools/nil plus simple constant expressions). In the C LLVM backend, GPU module constants are also resolved at compile time and GPU calls emit direct bridges to the pure C GPU API (`sgpu_*` in `gpu_api.h`), supporting both Vulkan and OpenGL backends. The bytecode VM provides 30 dedicated GPU opcodes for frame-loop hot paths.

The bytecode VM operates in hybrid mode by default: expressions, variables, loops (including break/continue), and function calls compile to stack bytecode, while unsupported constructs (classes, imports, exceptions, generators) fall back to the AST interpreter via `BC_OP_EXEC_AST_STMT`. This gives measurable speedups on loop-heavy workloads while maintaining full language coverage. Use `sage --runtime bytecode` or `sage --runtime auto` to enable.

1.4 Performance Characteristics

Sage ships with a benchmark suite (`benchmarks/01_fibonacci.sage` through `10_primes_sieve.sage`) with paired Python 3 implementations. Run `make benchmark-python` to compare. Typical results on the same workloads:

- **LLVM compiled**: 2-8x faster than CPython (fibonacci 3.9x, loop sum 7.7x)
- **C compiled**: 2-4x faster than CPython on most workloads

- **AST interpreter**: on par with CPython (faster on string/array ops, slower on recursion)
- **Bytecode VM**: similar to AST, faster on tight loops due to reduced dispatch overhead

---

Part 2: Internal Architecture and Core Modules

2.1 Module Dependency Graph

```
main.c
  |- lexer.c / lexer.h       [Tokenization, indentation tracking]
  |- parser.c / parser.h     [AST construction via recursive descent]
  |- interpreter.c / interpreter.h  [Tree-walking evaluation]
  |- value.c / value.h       [Runtime value representation]
  |- env.c / env.h           [Lexical scoping via linked-list environments]
  |- gc.c / gc.h             [Mark-and-sweep garbage collection]
  |- ast.c / ast.h           [AST node factory functions]
  |- token.h                 [Token type enumeration]
  |- module.c / module.h     [Module loading, caching, imports]
  |- compiler.c              [C code generation backend]
  |- llvm_backend.c          [LLVM IR generation (with GPU support)]
  |- llvm_runtime.c          [Standalone runtime for LLVM-compiled programs]
  |- codegen.c / codegen.h   [Native assembly (x86-64, aarch64, rv64, mips)]
  |- graphics.c / graphics.h [Vulkan GPU module for interpreter]
  |- gpu_api.c / gpu_api.h   [Pure C GPU API (Vulkan + OpenGL)]
  `- src/vm/                 [Bytecode VM: bytecode.c, vm.c, program.c, runtime.c]
```

2.2 Lexer (lexer.c / lexer.h)

Responsibility: Convert source text into a stream of **tokens** while tracking indentation levels (Python-style).

Key Data Structures: - **Token**: `{ TokenType type, const char* start, int length, int line }` - **Lexer State**: `{ const char* start, current, int line, int at_beginning_of_line }` - **Indent Stack**: Array tracking nesting depth; generates `TOKEN_INDENT` / `TOKEN_DEDENT` automatically

Token Types (from token.h): - **Keywords**: `and`, `as`, `async`, `await`, `break`, `case`, `catch`, `class`, `comptime`, `continue`, `default`, `defer`, `elif`, `else`, `end`, `enum`, `false`, `finally`, `for`, `from`, `if`, `import`, `in`, `init`, `let`, `macro`, `match`, `nil`, `not`, `or`, `print`, `proc`, `quote`, `raise`, `return`, `self`, `struct`, `super`, `trait`, `true`, `try`, `unquote`, `unsafe`, `var`, `while`, `yield`, `@` - *Note: **`print`**, **`end`**, **`match`**, **`init`**, **`enum`**, **`struct`**, and **`trait`** are soft keywords and can also be used as variable, property, or method names.* - **Operators**: `+`, `-`, `*`, `/`, `=`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `and`, `or`, `&`, `|`, `^`, `~`, `<<`, `>>` - **Punctuation**: `(`, `)`, `[`, `]`, `{`, `}`, `:`, `,`, `.` - **Structural**: `INDENT`, `DEDENT`, `NEWLINE`, `DOC_COMMENT`, `EOF`, `ERROR`

Indentation Handling (crucial for Python-like syntax):

Resource Management: - **SAGE_MAX_READ_SIZE**: A 100MB limit is enforced on all file I/O and network receive operations to prevent memory exhaustion DoS attacks.

- Tracks column position at the start of each line.
- Compares current indent to stack top:
  - **Increase**: Push new level, emit `TOKEN_INDENT`.
  - **Decrease**: Pop levels, emit `TOKEN_DEDENT` for each popped level.
  - **Mismatch**: Error if indent doesn't align with a previous level.

Example:

```
proc greet(name):
    let msg = "Hello, " + name
```

```
    print msg
# Tokens: proc, identifier(greet), (, identifier(name), ), :, newline,
#         indent, let, identifier(msg), =, string, newline,
#         print, identifier(msg), newline,
#         dedent, eof
```

2.3 Parser (parser.c)

Responsibility: Build an **Abstract Syntax Tree (AST)** from tokens using **recursive descent parsing**.

Parsing Levels (operator precedence):

```
expression      → assignment
assignment      → logical_or ("=" assignment)?
logical_or      → logical_and ("or" logical_and)*
logical_and     → equality ("and" equality)*
equality        → comparison (("==" | "!=") comparison)*
comparison      → addition (("<" | ">" | "<=" | ">=") addition)*
addition        → term (("+" | "-") term)*
term            → unary (("*" | "/") unary)*
unary           → ("-" unary) | postfix
postfix         → primary ("[" ... "]" | "." property)*
primary         → NUMBER | STRING | BOOL | NIL | "(" expr ")" | "[" array "]" | "{" dict "}" | IDENTIFI
```

Statement Parsing (top-down recursive descent):

```
statement   → if_stmt | while_stmt | for_stmt | return_stmt | break | continue | try_stmt | raise_stmt
declaration → class_decl | proc_decl | let_decl | statement
```

Key AST Node Types (see ast.h/ast.c): - **Expr** variants: `EXPR_NUMBER`, `EXPR_STRING`, `EXPR_BOOL`, `EXPR_NIL`, `EXPR_VARIABLE`, `EXPR_CALL`, `EXPR_BINARY`, `EXPR_UNARY`, `EXPR_INDEX`, `EXPR_SLICE`, `EXPR_ARRAY`, `EXPR_DICT`, `EXPR_TUPLE`, `EXPR_GET`, `EXPR_SET` - **Stmt** variants: `STMT_EXPRESSION`, `STMT_LET`, `STMT_PRINT`, `STMT_IF`, `STMT_WHILE`, `STMT_FOR`, `STMT_BLOCK`, `STMT_PROC`, `STMT_CLASS`, `STMT_RETURN`, `STMT_BREAK`, `STMT_CONTINUE`, `STMT_TRY`, `STMT_RAISE`, `STMT_YIELD`, `STMT_DEFER`, `STMT_MATCH`, `STMT_IMPORT`

Example Parsing:

```
Input:  let x = 10 + 5
Output: Stmt {
          type: STMT_LET,
          name: Token("x"),
          initializer: Expr {
            type: EXPR_BINARY,
            left: Expr { type: EXPR_NUMBER, value: 10 },
            op: Token(PLUS),
            right: Expr { type: EXPR_NUMBER, value: 5 }
          }
        }
```

2.4 Value System (value.h / value.c)

Responsibility: Define and manage **runtime value types** and their operations.

Core Value Type:

```c
struct Value {
  ValueType type;  // VAL_NUMBER, VAL_STRING, VAL_ARRAY, etc.
  union {
    double number;
```

```
    int boolean;
    char* string;
    NativeFn native;
    FunctionValue* function;
    ArrayValue* array;
    DictValue* dict;
    TupleValue* tuple;
    ClassValue* class_val;
    InstanceValue* instance;
    ExceptionValue* exception;
    GeneratorValue* generator;
  } as;
};
```

Value Types:

Type	Storage	Mutable	Use Case
`VAL_NUMBER`	`double` (on stack)	Yes	Integers, floats; `10`, `3.14`
`VAL_BOOL`	`int` (on stack)	Yes	Boolean logic; `true`, `false`
`VAL_NIL`	N/A	No	Null value; `nil`
`VAL_STRING`	`char*` (heap)	No	Text; immutable (assign new to mutate), supports bracket indexing (`str[i]`)
`VAL_ARRAY`	`ArrayValue*` (dynamic)	Yes	Lists; `[1, 2, 3]`, `push()`, indexing
`VAL_DICT`	`DictValue*` (string-keyed)	Yes	Maps; `{"key": value}`, `dict_set()`
`VAL_TUPLE`	`TupleValue*` (immutable)	No	Fixed-size sequences; `(1, 2, 3)`
`VAL_FUNCTION`	`FunctionValue*` (closure)	No	User-defined procs with captured environment
`VAL_NATIVE`	`NativeFn` (C function ptr)	No	Built-in functions like `len()`, `print()`
`VAL_CLASS`	`ClassValue*` (metadata)	No	Class definition with methods and parent
`VAL_INSTANCE`	`InstanceValue*` (fields dict)	Yes	Object; instance of a class
`VAL_EXCEPTION`	`ExceptionValue*` (message)	No	Exception; raised via `raise`
`VAL_GENERATOR` (`SAGE_TAG_GENERATOR`)	`GeneratorValue*` (resumable)	Yes	Iterable; maintains state across `yield`, supports native yield collector routines and `next()` builtin dispatch

Array Operations: - `array_push(arr, val)`: Append to dynamic array (resizes if needed). - `array_get(arr, index)`, `array_set(arr, index, val)`: Access/modify. - `array_slice(arr, start, end)`: Subarray (negative indices supported).

Dictionary Operations (O(1) amortized via hash table): - `dict_set(dict, key, value)`: Insert/update using FNV-1a hashing with open-addressing and linear probing. Table grows at 75% load factor. - `dict_get(dict, key)`, `dict_has(dict, key)`: O(1) average lookup. - `dict_delete(dict, key)`: Backward-shift deletion preserves probe chain integrity (no tombstones). - `dict_keys(dict)`, `dict_values(dict)`: Return as arrays (iterates capacity, skips empty slots).

String Operations: - `string_split(str, delim)`, `string_join(arr, sep)`: Splitting/joining arrays. - `string_replace(str, old, new)`, `string_upper/lower/strip(str)`: Transformation.

Class/Instance Operations: - `class_create(name, parent)`: Define a class with optional parent. - `class_add_method(class, name, method_stmt)`: Add methods. - `instance_create(class_def)`: Create instance with empty field dictionary. - `instance_set_field()`, `instance_get_field()`: Access instance variables.

2.5 Environment and Scoping (env.h / env.c)

Responsibility: Implement **lexical scoping** via a chain of nested environments (linked list).

Environment Structure:

```
struct Env {
  EnvNode* head;        // Variables defined in this scope (linked list)
  struct Env* parent;   // Enclosing scope
  struct Env* alloc_next; // Internal registry for GC/shutdown cleanup
  int marked;           // GC mark flag (0 = unmarked, 1 = reachable)
};

struct EnvNode {
  char* name;         // Variable name
  Value value;        // Stored value
  struct EnvNode* next;
};
```

Scoping Rules: - **`env_define(env, name, length, value)`**: Create or update a variable **in the current scope only**. If variable exists in current scope, update it; otherwise, create new. - **`env_get(env, name, length, value*)`**: Look up a variable, searching current scope then parent chains recursively. Return 1 if found, 0 if not. - **`env_assign(env, name, length, value)`**: Assign to an **existing** variable by searching up the scope chain. Used for reassigning already-defined variables.

Example Scoping:

```
Global: { x: 10, y: 20 }
  Block: { z: 30 } → parent: Global
    Inner: { x: 100 } → parent: Block

In Inner scope:
  - env_get("x") → 100 (found in current scope)
  - env_get("z") → 30 (found in parent Block)
  - env_get("y") → 20 (found in Global via parent chain)
```

2.6 Garbage Collector (gc.h / gc.c)

Responsibility: Automatically reclaim heap memory using tracing or reference counting.

GC Modes:

- **Tracing** (`--gc:tracing`, default): Concurrent tri-color mark-sweep with SATB write barriers. Best for general use with sub-millisecond pauses.
- **ARC** (`--gc:arc`): Deterministic reference counting. High performance for linear object graphs; does not collect cycles.
- **ORC** (`--gc:orc`): Optimized Reference Counting with Lins' trial deletion cycle collector. Combines the determinism of ARC with a robust cycle detection algorithm for complex graphs.

Tracing GC Overview: - **Mark Phase**: Starting from **root set** (global environment, function registry, call stack), mark all reachable objects. Born-black objects during concurrent marking. - **Sweep Phase**:

Free all unmarked objects and add them to a free pool. - **Triggering**: Automatic based on object count and heap pressure, or manual via `gc_collect()`.

GC Configuration:

```
#define GC_HEAP_SIZE (1024 * 1024)  // 1 MB theoretical heap
#define GC_THRESHOLD 1000           // Collect after 1000 object allocations
```

GC Header (prepended to all heap allocations):

```
struct GCHeader {
  int marked;    // 1 = marked (reachable), 0 = unmarked (candidate for freeing)
  int type;      // Object type (VAL_ARRAY, VAL_DICT, etc.)
  void* next;    // Linked list of all allocated objects
};
```

Root Set: - Global environment (`g_global_env`) and its entire scope chain. - Function registry (linked list of all defined `proc` declarations). - Call stack (environments of active function calls).

GC Statistics (via `gc_stats()` native function):

```
bytes_allocated: Total heap allocated (cumulative)
num_objects: Current number of live objects
collections: Number of GC runs so far
objects_freed: Objects freed in most recent collection
next_gc: Objects until next collection threshold
```

Usage:

```
# Automatic collection when threshold hit
let arr = [1, 2, 3]  # Allocates array

# Manual trigger
gc_collect()

# Check stats
let stats = gc_stats()
print stats
# Output: {"bytes_allocated": 1024, "num_objects": 5, "collections": 2, ...}

# Control GC
gc_disable()  # Pause automatic collection
gc_enable()   # Resume automatic collection
```

2.7 Interpreter (interpreter.c / interpreter.h)

Responsibility: **Tree-walking evaluation** of the AST, maintaining runtime state and executing statements/expressions.

Execution Result (tracks control flow):

```
struct ExecResult {
  Value value;           // Return value
  int is_returning;      // `return` statement hit
  int is_breaking;       // `break` statement hit
  int is_continuing;     // `continue` statement hit
  int is_throwing;       // Exception raised
  Value exception_value; // Exception object
  int is_yielding;       // `yield` hit (generators)
```

```
  void* next_stmt;       // Resume point for generators
};
```

Statement Execution (`interpret(stmt, env)`): - `STMT_LET`: Define variable (`let` or `var`) in current scope; evaluate initializer if present. - `STMT_EXPRESSION`: Evaluate expression for side effects. - `STMT_PRINT`: Evaluate expression and print value. - `STMT_IF`: Evaluate condition; execute then-branch or else-branch. - `STMT_WHILE`: Loop while condition is truthy; respect `break`/`continue`. - `STMT_FOR`: Iterate over array/range; update loop variable in scope. - `STMT_BLOCK`: Create child scope, execute statements in sequence. - `STMT_PROC`: Register function in function registry (not executed). - `STMT_CLASS`: Create class object; register methods. - `STMT_RETURN`: Exit function with value. - `STMT_TRY`: Execute try-block; on exception, search matching catch-clause; always run finally-block. - `STMT_RAISE`: Throw exception (string or exception object). - `STMT_YIELD`: In generators, pause execution and return value; next `next()` call resumes. - `STMT_IMPORT`: Load module; populate current environment with imports.

Expression Evaluation (`eval_expr(expr, env)`): - **Literals**: Numbers, strings, bools, nil returned as-is. - **Variables**: Look up in environment via `env_get()`. - **Binary Ops**: Evaluate left, then right (short-circuit for `and`/`or` in both interpreter and C backend); apply operator. - Arithmetic: `+`, `-`, `*`, `/` on numbers; `+` on strings (concatenation). - Comparison: `<`, `>`, `<=`, `>=` on numbers; `==`, `!=` on any type. - Logical: `and`, `or` with short-circuit evaluation (in both interpreter and C backend). - **Calls**: Look up function (user-defined or native); bind arguments to parameters; execute in new environment; return result. - **Indexing**: Evaluate array and index; return element or slice if range. - **Array/Dict/Tuple Construction**: Evaluate elements; construct heap-allocated structure. - **Property Access**: For instances, look up field in instance's field dict.

Function Call Mechanics: 1. Resolve callee to `ProcStmt*` or `NativeFn`. 2. Create **child environment** with current scope as parent. 3. Bind parameters to arguments via `env_define()`. 4. Execute function body statements. 5. On `return`, break with value; otherwise use `nil`. 6. Return to caller with result.

Generator Execution (Phase 7): 1. `proc my_gen():` creates a function that yields values. 2. Call `my_gen()` → returns `VAL_GENERATOR` value. 3. Call `next(gen)` → initializes generator environment, runs until first `yield`. 4. Each `next()` resumes from saved `current_stmt`; yields next value or `nil` if exhausted.

Native Function Registry: SageLang provides built-in functions injected into global environment via `init_stdlib(env)`:

Function	Signature	Purpose
`len(arr/str/dict)`	`Value → number`	Get length of collection
`push(arr, val)`	`(array, value) → nil`	Append to array
`pop(arr)`	`array → value`	Remove and return last element
`range(start, end)`	`(number, number) → array`	Generate list `[start, start+1, ..., end-1]`
`slice(arr/str, start, end)`	`(array/string, number, number) → array/string`	Extract subarray or substring
`split(str, delim)`	`(string, string) → array`	Split string by delimiter
`join(arr, sep)`	`(array, string) → string`	Join array elements with separator
`replace(str, old, new)`	`(string, string, string) → string`	Replace all occurrences
`upper(str)`	`string → string`	Uppercase
`lower(str)`	`string → string`	Lowercase
`strip(str)`	`string → string`	Trim whitespace
`str(val)`	`value → string`	Convert to string
`tonumber(str)`	`string → number`	Parse number
`input()`	`() → string`	Read line from stdin
`clock()`	`() → number`	Elapsed seconds since process start

Function	Signature	Purpose
`dict_keys(dict)`	`dict → array`	Get keys as array
`dict_values(dict)`	`dict → array`	Get values as array
`dict_has(dict, key)`	`(dict, string) → bool`	Check if key exists
`dict_delete(dict, key)`	`(dict, string) → nil`	Remove key from dict
`hasattr(obj, name)`	`(instance|dict, string) → bool`	Duck-typing check for instance methods, fields, and dictionary keys
`val_tag(val)`	`value → number`	Get internal numeric tag identifier of a value
`build_info()`	`() → dict`	Get binary build metadata (version, arch, build type, spec version)
`type(val)`	`value → string`	Get string name of type (type(val) returns "bytes" for byte buffers)
`chr(n)`	`number → string`	Get single-character string from ASCII code
`ord(c)`	`string → number`	Get ASCII code of single-character string
`hash(val)`	`value → number`	Get hash code of a value
`sizeof(val)`	`value → number`	Get memory size of a value in bytes
`doc(val)`	`value → string`	Retrieve documentation from a function
`gc_collect()`	`() → nil`	Trigger garbage collection
`gc_stats()`	`() → dict`	Get GC statistics
`gc_enable()`, `gc_disable()`	`() → nil`	Control automatic GC
`next(gen)`	`generator → value`	Resume generator, get next yielded value
`ffi_open(path)`	`string → clib`	Open shared library via dlopen
`ffi_call(lib, func, ret, ...)`	`(clib, string, string, ...) → value`	Call C function in library
`ffi_sym(lib, name)`	`(clib, string) → bool`	Check if symbol exists in library
`ffi_close(lib)`	`clib → nil`	Close shared library handle
`mem_alloc(size)`	`number → pointer`	Allocate raw memory (zero-initialized)
`mem_free(ptr)`	`pointer → nil`	Free allocated memory
`mem_read(ptr, off, type)`	`(pointer, number, string) → value`	Read value at ptr+offset
`mem_write(ptr, off, type, val)`	`(pointer, number, string, value) → nil`	Write value at ptr+offset
`mem_size(ptr)`	`pointer → number`	Get allocation size
`addressof(val)`	`value → number`	Get memory address of a value
`asm_exec(code, ret, ...)`	`(string, string, ...) → value`	Compile and execute assembly
`asm_compile(code, arch, out)`	`(string, string, string) → bool`	Cross-compile assembly to object file
`asm_arch()`	`() → string`	Get host architecture name
`struct_def(fields)`	`array → dict`	Define C struct layout with alignment

Function	Signature	Purpose
`struct_new(def)`	`dict → pointer`	Allocate zeroed struct instance
`struct_get(ptr, def, name)`	`(pointer, dict, string) → value`	Read struct field
`struct_set(ptr, def, name, val)`	`(pointer, dict, string, value) → nil`	Write struct field
`struct_size(def)`	`dict → number`	Get total struct size
`cpu_count()`	`() → number`	Logical CPU core count
`cpu_physical_cores()`	`() → number`	Physical CPU core count
`cpu_has_hyperthreading()`	`() → bool`	HT check
`thread_set_affinity(id)`	`number → number`	Pin current thread to core
`thread_get_core()`	`() → number`	Current CPU core ID
`atomic_new(val)`	`value → pointer`	Create atomic long
`atomic_load(a)`	`pointer → number`	Atomic read
`atomic_store(a, v)`	`(pointer, number) → nil`	Atomic write
`atomic_add(a, v)`	`(pointer, number) → number`	Atomic fetch-and-add
`atomic_cas(a, e, d)`	`(pointer, num, num) → bool`	Atomic compare-and-swap
`atomic_exchange(a, v)`	`(pointer, number) → number`	Atomic fetch-and-exchange
`sem_new(n)`	`number → pointer`	Create POSIX semaphore
`sem_wait(s)`	`pointer → nil`	Blocking wait
`sem_post(s)`	`pointer → nil`	Signal/release
`sem_trywait(s)`	`pointer → bool`	Non-blocking wait

2.8 Module System (module.h / module.c)

Responsibility: Load `.sage` files as reusable modules and manage imports.

Module Structure:

```c
struct Module {
  char* name;               // Module identifier (e.g., "math")
  char* path;               // Full filesystem path to .sage file
  Environment* env;         // Module's own environment (exports)
  bool is_loaded;           // Execution complete?
  bool is_loading;          // Currently executing? (cycle detection)
  struct Module* next;      // For linked-list cache
};

struct ModuleCache {
  Module* modules;          // All loaded modules
  char** search_paths;      // Directories to search (., ./lib, ./modules)
  int search_path_count;
};
```

Module Lifecycle: 1. **`load_module(cache, "math")`**: Search in `search_paths` for `math.sage` or `math/__init__.sage`; create `Module` struct; add to cache. 2. **`execute_module(module, global_env)`**: Read file; lex/parse/interpret in module's own environment (child of global). 3. **Exports**: Any top-level `proc`, `class`, `let` in module becomes available for import.

Import Statements (parsed in `parser.c`, executed in `interpreter.c`):

```
# Import entire module (unaliased)
import math
# → Available as: math (currently placeholder; future: module.sin(), etc.)
```

```
# Import module with alias
import math as m
# → Available as: m

# Import specific items
from math import sin, cos
# → sin and cos directly available in scope

# Import specific items with aliases
from math import sin as sine, cos as cosine
# → sine, cosine available in scope

# Star import (all exports)
from math import *
# → All module exports available directly
```

Search Path Resolution: 1. Try `{search_path}/module_name.sage`. 2. Try `{search_path}/module_name/__init__.sa
3. If not found in any search path, error.

Default Search Paths:

```
- "."           (current directory)
- "./lib"       (local lib subdirectory)
- "./modules"   (local modules subdirectory)
```

Import Processing: - `import_all(env, "math")`: Load module, define module name in env. - `import_from(env, "math", items[], count)`: Load module, look up items, define in env. - `import_as(env, "math", "m")`: Load module, define alias in env.

Compiled LLVM Constant Imports: - Applies to both LLVM pipelines: the C backend (`src/c/llvm_backend.c`) and the self-hosted backend (`src/sage/llvm_backend.sage`). - `from module import NAME` (and `from module import NAME as ALIAS`) is pre-resolved from module top-level `let` constants when they are compile-time foldable. - Supported imported value shapes are scalar constants (`number`, `string`, `bool`, `nil`) and expressions composed from previously resolved constants. - Search paths for this compile-time resolution match module lookup defaults: `./`, `./lib`, and `./modules`. - In the C LLVM backend, `from gpu import CONST` is also resolved from the built-in GPU constant table. - Unresolved imported constants are treated as compile errors in LLVM codegen rather than generating unresolved `%NAME` loads.

Circular Dependency Detection: `is_loading` flag prevents infinite loops during module execution.

Part 3: Language Features by Phase

Phase 1–3: Core Language (Arithmetic, Variables, Control Flow)

Implemented: Basic arithmetic, variable declaration, conditionals, loops, functions.

Example:

```
let x = 10
let y = 20
print x + y  # Output: 30

if x > 5:
    print "x is greater than 5"
else:
    print "x is not greater than 5"
```

```
proc add(a, b):
    return a + b

print add(3, 4)  # Output: 7
```

Phase 4: Garbage Collection

Added: Mark-and-sweep GC automatically manages heap.

Example:

```
let arr = [1, 2, 3, 4, 5]
print len(arr)          # 5

# GC runs automatically when threshold hit
gc_collect()            # Force collection
let stats = gc_stats()
print stats["collections"]  # Number of GC runs
```

Phase 5–6: Data Structures and OOP

Phase 5: Arrays, dicts, tuples, slicing, indexing.

```
let arr = [1, 2, 3, 4, 5]
print arr[0]            # 1
print arr[1:3]          # [2, 3]
arr[0] = 10
print arr               # [10, 2, 3, 4, 5]

let dict = {"name": "Alice", "age": 30}
print dict["name"]      # "Alice"
dict["age"] = 31

let tuple = (1, 2, 3)
print tuple             # (1, 2, 3)  # Immutable
```

Phase 6: Classes, inheritance, instance methods.

```
class Animal:
    proc init(name):
        self.name = name
        self.age = 0

    proc birthday():
        self.age = self.age + 1

    proc speak():
        print self.name + " makes a sound"

class Dog(Animal):
    proc speak():
        print self.name + " barks"

let dog = Dog("Buddy")
print dog.name           # "Buddy"
dog.birthday()
```

```
print dog.age            # 1
dog.speak()              # "Buddy barks"
```

Phase 7: Advanced Control Flow, Exceptions, Generators

Exceptions:

```
proc divide(a, b):
    if b == 0:
        raise "Division by zero"
    return a / b

try:
    print divide(10, 0)
catch e:
    print "Caught error: " + e
finally:
    print "Cleanup done"
```

Generators:

```
proc count_up(n):
    let i = 0
    while i < n:
        yield i
        i = i + 1

let gen = count_up(3)
print next(gen)         # 0
print next(gen)         # 1
print next(gen)         # 2
print next(gen)         # nil (exhausted)
```

Deferred Code (Phase 7 addition): `defer` executes a statement on scope exit, in LIFO (Last-In, First-Out) order, which is extremely useful for cleanup:

```
proc read_and_print():
    let f = file_open("data.txt")
    defer file_close(f)
    print file_read(f)
```

Pattern Matching (Phase 7 addition): `match` allows matching a value against multiple patterns, with optional guards and a default fallback:

```
let x = 42
match x:
    case 1:
        print "one"
    case 42 if y > 0:
        print "forty-two with positive y"
    case 42:
        print "forty-two"
    default:
        print "other"
```

Phase 8: Modules

```
# math.sage
proc sin(x):
    # Approximate sine
    return x  # Stub

proc cos(x):
    return 1  # Stub

# main.sage
import math as m
print m.sin(0)  # Calls math.sin in module's namespace

# Or:
from math import sin, cos
print sin(0)
print cos(0)
```

Phase 9: Low-Level Programming

Bitwise Operators:

```
# Bitwise AND, OR, XOR
print 5 & 3       # 1  (0101 & 0011 = 0001)
print 5 | 3       # 7  (0101 | 0011 = 0111)
print 5 ^ 3       # 6  (0101 ^ 0011 = 0110)

# Bitwise NOT
print ~0          # -1 (all bits flipped)

# Shift operators
print 1 << 4      # 16 (shift left by 4)
print 16 >> 2     # 4  (shift right by 2)

# Practical: extract lower nibble
let val = 255
let mask = 15
print val & mask   # 15

# Practical: check if bit is set
let x = 7
let bit2 = (x >> 2) & 1
print bit2 == 1    # true
```

Bitwise Safety: Shift amounts are validated at runtime — values outside 0-63 return 0 instead of causing C undefined behavior. Right-shift is arithmetic (sign-extending) on signed values. Floating-point operands are truncated to `long long` before bitwise operations.

Unsafe Blocks: As of version 4.0.1 (Specification 2.0), low-level memory operations or operations bypassing typical safety checks should be wrapped in an `unsafe` block. `unsafe` blocks must be terminated with the `end` keyword.

```
unsafe:
    # Memory primitives or FFI calls here
    let ptr = mem_alloc(16)
    mem_write(ptr, 0, "int", 42)
```

```
    mem_free(ptr)
end
```

Metaprogramming (Phase 17): SageLang includes experimental metaprogramming syntax constructs evaluated during parsing and compilation: - `comptime(expr)`: Evaluates the inner expression at compile-time (constant folding). - `comptime:`: A block statement executing code directly within the compiler context. - `macro name(args):`: Declares an AST transformation macro. - `quote` / `unquote`: Reserved lexer keywords for future AST token quoting integration.

Note: Phase 17 features are implemented in the parser but have incomplete compiler backend propagation outside constant expressions.

Foreign Function Interface (FFI):

```
# Open a shared C library
let libm = ffi_open("libm.so.6")

# Call C functions with return type and arguments
let result = ffi_call(libm, "sqrt", "double", 144.0)
print result       # 12

let s = ffi_call(libm, "sin", "double", 0.0)
print s            # 0

# Check if a symbol exists
print ffi_sym(libm, "cos")    # true
print ffi_sym(libm, "bogus")  # false

# Close when done
ffi_close(libm)

# Call libc functions
let libc = ffi_open("libc.so.6")
let n = ffi_call(libc, "strlen", "long", "hello")
print n            # 5
ffi_close(libc)
```

FFI supports return types: `"double"`, `"int"`, `"long"`, `"string"`, `"void"`, with up to 3 arguments (numeric or string). Passing more than 3 arguments returns an error. Library handles are tracked by the GC and properly freed on collection.

Raw Memory Operations:

```
# Allocate 32 bytes of raw memory
let buf = mem_alloc(32)
print mem_size(buf)    # 32

# Write and read different types
mem_write(buf, 0, "byte", 42)
print mem_read(buf, 0, "byte")     # 42

mem_write(buf, 4, "int", 12345)
print mem_read(buf, 4, "int")      # 12345

mem_write(buf, 8, "double", 3.14)
print mem_read(buf, 8, "double")   # 3.14
```

```
# Get memory address of any value
let arr = [1, 2, 3]
print addressof(arr)   # memory address as number

# Free when done
mem_free(buf)
```

The `mem_read(ptr, offset, type)` function requires exactly 3 arguments, and `mem_write(ptr, offset, type, value)` requires exactly 4 arguments. Supported explicit type strings for `mem_read`/`mem_write` are: `"byte"` (1 byte), `"int"` (4 bytes), `"double"` (8 bytes), `"string"` (read-only, null-terminated). Allocations are capped at 64MB. Negative offsets are rejected. Bounds checking is enforced for owned memory (offset + type size must not exceed allocation). Double-free is prevented via handle nullification. Memory pointers are GC-tracked and freed on collection if owned.

Inline Assembly (x86-64, aarch64, rv64, mips):

```
# Detect host architecture
print asm_arch()           # "x86_64"

# Execute x86-64 assembly: return a constant
let val = asm_exec("    mov $42, %rax", "int")
print val                  # 42

# Add two integers (args in rdi, rsi per System V ABI)
let sum = asm_exec("    mov %rdi, %rax\n    add %rsi, %rax", "int", 10, 32)
print sum                  # 42

# Add two doubles (args in xmm0, xmm1)
let r = asm_exec("    addsd %xmm1, %xmm0", "double", 1.5, 2.7)
print r                    # 4.2

# Cross-compile for aarch64 (requires aarch64-linux-gnu-as)
let ok = asm_compile("    mov x0, #42", "aarch64", "/tmp/out.o")

# Cross-compile for RISC-V 64 (requires riscv64-linux-gnu-as)
let ok2 = asm_compile("    li a0, 42", "rv64", "/tmp/out_rv.o")
```

Supported architectures: `"x86_64"`, `"aarch64"`, `"rv64"`, `"mips"`. Return types: `"int"`, `"double"`, `"void"`. Up to 4 numeric arguments.

C Struct Interop:

```
# Define a C-compatible struct: { int x; int y; double z; }
let Point = struct_def([["x", "int"], ["y", "int"], ["z", "double"]])
print struct_size(Point)   # 16 (with alignment)

# Allocate and populate
let p = struct_new(Point)
struct_set(p, Point, "x", 10)
struct_set(p, Point, "y", 20)
struct_set(p, Point, "z", 3.14)

# Read fields
print struct_get(p, Point, "x")   # 10
print struct_get(p, Point, "y")   # 20
print struct_get(p, Point, "z")   # 3.14
```

```
mem_free(p)

# Alignment example: { char a; double b; int c; }
# Layout: a@0, pad(7), b@8, c@16, pad(4) = 24 bytes
let S = struct_def([["a", "char"], ["b", "double"], ["c", "int"]])
print struct_size(S)              # 24
```

Supported types: `"char"`, `"byte"` (1), `"short"` (2), `"int"` (4), `"long"` (8), `"float"` (4), `"double"` (8), `"ptr"` (8). Alignment follows C ABI rules.

Part 4: Writing SageLang Programs

4.1 Basic Syntax Patterns

Variables and Types:

```
let x = 42                      # Variable binding
var y = 100                     # Explicitly mutable binding
let s = "Hello, World"          # String
let b = true                    # Boolean
let arr = [1, 2, 3]             # Array
let dict = {"a": 1, "b": 2}     # Dictionary
let tup = (1, "two", 3)         # Tuple (immutable)
let n = nil                     # Nil (null)
```

Operators:

```
# Arithmetic
print 10 + 5           # 15
print 10 - 5           # 5
print 10 * 5           # 50
print 10 / 2           # 5
print -5               # -5

# Comparison
print 5 == 5           # true
print 5 != 3           # true
print 5 > 3            # true
print 5 < 10           # true

# Logical
print true and false   # false
print true or false    # true

# Bitwise
print 5 & 3            # 1
print 5 | 3            # 7
print 5 ^ 3            # 6
print ~0               # -1
print 1 << 4           # 16
print 16 >> 2          # 4
```

String Operations:

```
let a = "Hello"
print a[0]                    # "H" (Indexing returns a single-character string)
```

```
print indexof("hello", "l")    # 2 (Accepts exactly 2 strings, returns index or -1)
let b = "World"
print a + " " + b              # "Hello World"

let words = "a,b,c".split(",")
print words                     # ["a", "b", "c"]

let joined = ["x", "y"].join("-")
print joined                    # "x-y"

print "hello".upper()          # "HELLO"
print "HELLO".lower()          # "hello"
```

Arrays:

```
let arr = [10, 20, 30, 40, 50]
print len(arr)                 # 5
print arr[0]                   # 10
arr[0] = 100
print arr[0]                   # 100
push(arr, 60)
print len(arr)                 # 6

# Slicing
print arr[1:3]                 # [20, 30]
print arr[2:]                  # [30, 40, 50, 60] (from index 2 to end)
print arr[:2]                  # [100, 20] (from start to index 2)
```

Dictionaries:

```
let dict = {"name": "Alice", "age": 30}
print dict["name"]             # "Alice"
dict["age"] = 31
print dict_has(dict, "name")   # true
print dict_keys(dict)          # ["name", "age"]
print dict_values(dict)        # ["Alice", 31]

dict_delete(dict, "age")
print dict_has(dict, "age")    # false
```

4.2 Control Flow Patterns

Conditionals:

```
let x = 15

if x < 10:
    print "small"
else:
    print "large"

# Nested conditions
if x > 10:
    if x > 20:
        print "very large"
    else:
        print "medium"
```

```
else:
    print "small"
```

Loops:

```
# While loop
let i = 0
while i < 5:
    print i
    i = i + 1

# For loop over array
let arr = [10, 20, 30]
for item in arr:
    print item

# For loop over range
for i in range(0, 5):
    print i

# Note: Strings are not currently iterable via `for` loops.
# Use a `while` loop with index-based access instead.
let s = "hello"
let i_str = 0
while i_str < len(s):
    print s[i_str]
    i_str = i_str + 1

# Break and continue
for i in range(0, 10):
    if i == 3:
        continue
    if i == 7:
        break
    print i
```

4.3 Functions and Closures

Function Definition and Call:

```
proc greet(name):
    print "Hello, " + name

greet("Alice")

proc add(a, b):
    return a + b

let result = add(5, 3)
print result                    # 8
```

Closures (captured environment):

```
proc make_multiplier(factor):
    proc multiply(x):
        return x * factor     # Captures 'factor' from enclosing scope
    return multiply
```

```
let times3 = make_multiplier(3)
let times5 = make_multiplier(5)

print times3(10)              # 30
print times5(10)              # 50
```

4.4 Structs, Enums, Traits, and Classes

Struct Definition (Phase 1.7): SageLang provides native `struct` support as a lightweight alternative to classes. Structs automatically generate `init`, `to_string`, and equality methods based on their fields. Under the hood, structs are represented as standard class/instance values (`VAL_CLASS`/`VAL_INSTANCE`).

```
struct Point:
    x
    y
end

let p = Point(10, 20)
print p.x           # 10
print p             # Point(x=10, y=20)
```

Enum Definition (Phase 1.7): Native C-like `enum` support maps variant names to sequential integers starting from 0. At runtime, enums are represented as dictionaries. Note that tagged unions (ADTs with associated data) are still implemented via the standard library (`std.enum`).

```
enum Color:
    Red
    Green
    Blue

print Color.Red     # 0
print Color.Green   # 1
```

Trait Definition (Phase 1.7): Traits define an interface contract of method signatures. A trait compiles down to a dictionary containing the list of required method names.

```
trait Printable:
    proc to_string(self)
end
```

Class Definition:

```
class Point:
    proc init(x = 0, y = 0):  # Default parameters are supported
        self.x = x
        self.y = y

    proc distance_from_origin():
        # Simplified (no sqrt)
        return self.x * self.x + self.y * self.y

let p = Point(3, 4)
print p.x                      # 3
print p.y                      # 4
print p.distance_from_origin() # 25
```

Inheritance:

```
class Vehicle:
    proc init(wheels):
        self.wheels = wheels

    proc describe():
        print "Vehicle with " + str(self.wheels) + " wheels"

class Car(Vehicle):
    proc init(wheels, doors):
        self.wheels = wheels
        self.doors = doors

    proc describe():
        print "Car with " + str(self.wheels) + " wheels and " + str(self.doors) + " doors"

let car = Car(4, 4)
car.describe()                 # "Car with 4 wheels and 4 doors"
```

Calling Parent Methods with `super`:

Use `super.init(args)` to call the parent class constructor, and `super.method(args)` to call any parent method. `self` is automatically injected. This works with chained inheritance (3+ levels, e.g., A → B → C).

```
class Animal:
    proc init(self, name):
        self.name = name
    proc speak(self):
        print self.name + " speaks"

class Dog(Animal):
    proc init(self, name, breed):
        super.init(name)
        self.breed = breed
    proc speak(self):
        super.speak()
        print self.name + " barks"

let d = Dog("Rex", "Labrador")
d.speak()
# Rex speaks
# Rex barks
```

The `->` arrow operator can also be used with super: `super->init(args)`.

Arrow Operator (`->`):

The `->` operator is a syntactic alias for `.` (dot), providing systems-language style pointer/member access. It works identically for field reads, field writes, and method calls.

```
let p = Point(3, 7)
print p->x          # same as p.x
p->x = 10           # same as p.x = 10
print p->to_string()  # same as p.to_string()
```

`->` is interchangeable with `.` everywhere: attribute access, method calls, and `super` calls (`super->init(self, args)`).

4.5 Exception Handling

Try-Catch-Finally:

```
proc safe_divide(a, b):
    if b == 0:
        raise "Zero denominator"
    return a / b

try:
    let result = safe_divide(10, 0)
catch error:
    print "Error: " + error
finally:
    print "Done"

# Output:
# Error: Zero denominator
# Done
```

Finally Block Semantics: The finally block always executes, and its control flow takes precedence over try/catch. If finally contains `return`, `break`, `continue`, or `raise`, that overrides the try/catch result. If finally executes normally, the try/catch result is preserved. This matches Python/Java behavior.

Raise: You can raise any value. Strings become exception messages directly. Numbers, booleans, and nil are converted to their string representation. Non-string/non-exception values become "Unknown error".

```
raise "file not found"         # Exception with message "file not found"
raise 404                      # Exception with message "404"
raise nil                      # Exception with message "nil"
```

4.6 Generators

Simple Generator:

```
proc fibonacci(n):
    let a = 0
    let b = 1
    let count = 0
    while count < n:
        yield a
        let temp = a + b
        a = b
        b = temp
        count = count + 1

let gen = fibonacci(5)
print next(gen)                # 0
print next(gen)                # 1
print next(gen)                # 1
print next(gen)                # 2
print next(gen)                # 3
print next(gen)                # nil
```

4.7 Modules

math.sage:

```
let PI = 3.14159

proc square(x):
    return x * x

proc cube(x):
    return x * x * x

proc factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

main.sage:

```
import math as m

print m.PI                      # 3.14159
print m.square(5)               # 25
print m.cube(3)                 # 27
print m.factorial(5)            # 120

# Or:
from math import PI, square, cube
print PI
print square(5)
print cube(3)
```

4.8 Defer Statements

Defer schedules cleanup code to run when the enclosing block exits, regardless of whether exit is via normal flow, `return`, `break`, `continue`, or exception. Multiple defers execute in LIFO (last-in, first-out) order.

```
proc process_file(path):
    let handle = open(path)
    defer:
        close(handle)
        print("file closed")
    # ... work with handle ...
    # close() runs automatically when block exits

proc multi_defer():
    defer:
        print("third")
    defer:
        print("second")
    defer:
        print("first")
    print("working")
    # Output: working, first, second, third
```

Defer works with all exit paths: - Normal block completion - `return` from a function - `break`/`continue` from a loop - Exceptions (defers run before exception propagates)

Implementation: the interpreter collects deferred statements in a 1024-slot stack per block. On block exit, they execute in reverse order.

4.9 Pattern Matching (match/case/default)

Match evaluates a value and compares it against case patterns using equality. The first matching case body executes. An optional `default` clause runs if no case matches.

```
let cmd = "hello"
match cmd:
    case "quit":
        print("quitting")
    case "hello":
        print("greeting")
    case "help":
        print("showing help")
    default:
        print("unknown command")

# Output: greeting
```

Match works with numbers, strings, booleans, and nil:

```
proc classify(n):
    match n:
        case 1:
            return "small"
        case 2:
            return "medium"
        default:
            return "big"

print(classify(1))    # small
print(classify(100))  # big
```

Cases are checked in order using value equality (`==`). Match is supported in the interpreter, C compiler (emits if/else-if chain), LLVM backend (emits branch instructions), and native codegen.

Part 5: Implementation Details and Runtime Behavior

5.1 Memory Layout and Object Representation

Stack vs. Heap: - **Stack-allocated** (cheap, temporary): `Value` struct for numbers, bools, nil, function pointers. - **Heap-allocated** (managed by GC): Arrays, dicts, tuples, strings, classes, instances, exceptions, generators.

Value Encoding (union-based):

```
Value v;
v.type = VAL_NUMBER;
v.as.number = 42.5;        // Efficient packing; only one field used per type

// Or:
v.type = VAL_ARRAY;
v.as.array = malloc(...);  // Pointer to ArrayValue struct
```

Implications: - Numbers and bools are always "cheap" to copy (no allocation). - Strings are immutable; reassignment creates new allocation (handled by parser/interpreter). - Arrays/dicts are mutable and passed by reference (modifications visible across scopes).

5.2 Call Stack and Environments

On Function Call: 1. **Create child environment**: `new_env = env_create(current_env)`. 2. **Bind parameters**: `env_define(new_env, param_name, param_value)`. 3. **Execute body**: `interpret(body, new_env)`. 4. **Return**: Pop environment, return to caller.

Nested Example:

```
Global: { x: 10 }
  `- call add(3, 4)
      `- Local: { a: 3, b: 4 } (parent: Global)
         `- return a + b (7)
```

5.3 Exception Propagation

Raise Mechanism: - `raise "message"` or `raise exception_obj` → Sets `is_throwing = 1` in `ExecResult`. - `raise` converts non-string values: numbers become their string representation, booleans become "true"/"false", nil becomes "nil". - All statements/expressions check `is_throwing` and propagate up. - First matching `catch` clause executes; if none, exception exits function. - Sage uses generic catch (no type-based matching). All catches handle all exceptions.

Finally Guarantee: - `finally` block always runs, even if exception, return, break, or continue occurred in try/catch. - If `finally` raises an exception or returns, that overrides the try/catch result (matches Python/Java semantics). - If `finally` completes normally, the original try/catch result is preserved.

Memory Safety: - Exception messages are allocated outside the GC heap but tracked via `gc_track_external_allocation()` - GC properly frees exception messages with `gc_track_external_free()` during collection. - VAL_EXCEPTION objects are marked during GC mark phase to prevent premature collection.

5.4 Generator State Management

Creation: - `proc gen_func(): yield 1; yield 2` parses as normal proc. - **Call** returns `VAL_GENERATOR` with: - `body`: Pointer to function body (Stmt*). - `closure`: Captured environment at call time. - `is_started`: 0 initially. - `is_exhausted`: 0 initially. - `current_stmt`: NULL (set on first `next()`).

Resumption: - **First `next()`**: Initialize `gen_env` as child of closure, run body until first `yield`. - **Subsequent `next()`**: Resume from `current_stmt` (set by prior yield), run until next `yield` or end. - **Yield execution**: Save `current_stmt = stmt->next`, return yielded value with `is_yielding = 1`.

Iteration Limits: - Direct `for-in` iteration over generators is explicitly rejected with a runtime error (`Runtime Error: 'Generator' object is not iterable.`). Always use `next()` for extraction.

Example Trace:

```
gen = my_gen()                    # Create generator, is_started=0
next(gen)                          # Initialize gen_env, run to first yield
                                   # Yield 10; current_stmt = stmt after yield
next(gen)                          # Resume from current_stmt, run to next yield
                                   # Yield 20; current_stmt = stmt after yield
next(gen)                          # Resume, reach end, is_exhausted=1
                                   # Return nil
```

5.5 Operator Precedence and Associativity

Precedence (lowest to highest): 1. Assignment (`=`) 2. Logical OR (`or`) 3. Logical AND (`and`) 4. Bitwise OR (`|`) 5. Bitwise XOR (`^`) 6. Bitwise AND (`&`) 7. Equality (`==`, `!=`) 8. Comparison (`<`, `>`, `<=`, `>=`) 9. Shift (`<<`, `>>`) 10. Addition/Subtraction (`+`, `-`) 11. Multiplication/Division (`*`, `/`, `%`) 12. Unary (`-`, `not`, `~`) 13. Postfix (indexing, slicing, property access) 14. Primary (literals, variables, parens, function calls)

Associativity: - **Left**: Most binary operators (`+`, `-`, `*`, `/`, `==`, `!=`, `<`, `>`, etc.). - **Right**: Assignment (`=`), unary (`-`).

5.6 Type Coercion and Truthiness

Truthiness (for `if`/`while` conditions): - **Falsy**: `nil`, `false`, `0`, and empty strings (`""`). - **Truthy**: Everything else (empty arrays are truthy, objects, etc.).

Type Coercion: - **Addition**: If both are numbers, add; if either is string, concatenate; else nil. - **Comparison**: Numbers compared numerically; strings compared lexically; different types are unequal. - **String conversion**: `str()` native function converts any value to string.

Part 6: Compilation and Execution

6.1 Building the Interpreter

The repository exposes four build paths from source:

- Desktop build from C sources, which produces `sage` and `sage-lsp`
- Self-hosted bootstrap mode, which still builds `sage` first and then uses it to execute `src/sage/sage.sage`
- Pico/RP2040 builds through CMake and the Pico SDK
- SageMake: unified build system with auto-detection of platform, GPU (cuBLAS), NPU (NNAPI/SNPE/ONE), SIMD (NEON/RVV), and compiler backends

Desktop Build (Make):

```
make clean && make -j$(nproc)
./sage examples/hello.sage
```

ML Trainer Build:

```
make train-c      # Builds standalone C training binary (no Sage runtime); uses src/c/ml_backend.c dire
```

Desktop Build (CMake):

```
cmake -B build
cmake --build build
```

SageMake Build:

```
./sagemake info              # Show detected environment (GPU, NPU, SIMD, compiler)
./sagemake build             # Build sage interpreter
./sagemake chatbot --llvm    # Compile chatbot via LLVM
./sagemake chatbot --c       # Compile via C backend
./sagemake chatbot --native  # Compile via native asm
./sagemake train 200000 0.001  # Build trainer + train
./sagemake all               # Build everything
./sagemake --minimal build   # Core only (no optional deps)
```

`./sagemake info` auto-detects GPU (cuBLAS), NPU (NNAPI/SNPE/ONE), SIMD (NEON/RVV), and selects optimal compilation flags.

Self-Hosted Build / Bootstrap:

```
make sage-boot FILE=examples/hello.sage
make test-selfhost

# Or via CMake:
cmake -B build_sage -DBUILD_SAGE=ON
```

```
cmake --build build_sage
cmake --build build_sage --target test_selfhost
```

Pico Build:

```
cmake -B build_pico -DBUILD_PICO=ON -DPICO_BOARD=pico
cmake --build build_pico
```

Desktop builds require `libcurl` and OpenSSL development headers/libraries in addition to a C compiler, `make`, and/or `cmake`.

3 Build Parameter Reference Make Variables:

Variable	Default	Effect
`CC`	`gcc`	C compiler used by `make`
`CFLAGS`	`-std=c11 -Wall -Wextra -Wpedantic -O2 -D_POSIX_C_SOURCE=200809L`	Base compile flags for desktop builds
`LDFLAGS`	`-lm -lpthread -ldl -lcurl -lssl -lcrypto`	Desktop link flags; reduced to `-lm` when `PICO_BUILD` is set
`DEBUG`	`0`	`DEBUG=1` adds `-g -O0 -DDEBUG`
`PREFIX`	`/usr/local`	Install prefix used by `make install`
`FILE`	unset	Required by `make sage-boot FILE=<path>`
`PICO_BUILD`	unset	Internal Make switch that changes link flags for non-desktop builds

CMake Cache Variables / Environment Inputs:

Parameter	Default	Effect
`BUILD_PICO`	`OFF`	Enables Pico/RP2040 output and imports `pico_sdk_import.cmake` before `project()`
`BUILD_SAGE`	`OFF`	Enables bootstrap/self-hosted targets such as `sage_boot` and `test_selfhost`
`ENABLE_DEBUG`	`OFF`	Adds `-g -O0 -DDEBUG`
`ENABLE_TESTS`	`OFF`	Builds optional C test executables
`CMAKE_BUILD_TYPE`	generator default	Standard CMake build-type selector
`CMAKE_C_COMPILER`	toolchain default	Chooses the C compiler reported in the configuration summary
`CMAKE_INSTALL_PREFIX`	CMake default	Install destination for `cmake --install`

Parameter	Default	Effect
PICO_SDK_PATH	unset	Required for Pico builds unless already exported in the environment
PICO_BOARD	pico	Pico board name for SDK builds
SAGE_FILE	unset	Input path consumed by the sage_boot custom target

6.2 Main Entry Point (main.c)

src/c/main.c initializes the garbage collector, registers raw argv for the sys module, initializes the module cache, and then dispatches one of the top-level modes below.

3 sage CLI Parameter Reference

Command	Meaning	Notes
sage	Start the interactive REPL	Same as sage --repl
sage --repl	Start the interactive REPL	Supports multi-line blocks and recovery after REPL errors
sage --help	Print usage text	Covers compiler, tooling, and REPL entry points
sage -c "source"	Execute a source string	No file is loaded
sage <file.sage> [arg ...]	Execute a Sage file	Additional CLI arguments are visible through sys.args()
sage <file.sgvm>	Execute an SGVM binary	Runs via integrated MetalVM engine
sage --lsp	Start the LSP server on stdin/stdout	sage-lsp is the standalone wrapper binary
sage fmt <file>	Format a file in place	Prints Formatted: <file> on success
sage fmt --check <file>	Check formatting without rewriting	Exit code 1 when changes are needed
sage lint <file>	Run the static linter	Exit code 1 when issues are found

Compiler Command	Default Output	Options
sage --emit-vm <input.sage>	<input>.svm	-o <path>, -O0, -O1, -O2, -O3, -g
sage --sgvm <input.sage>	<input>.sgvm	-o <path>, -O0, -O1, -O2, -O3, -g
sage --compile <input.sage>	<input-without-.sage>	-o <path>, --cc <compiler>, -O0, -O1, -O2, -O3, -g
sage --emit-llvm <input.sage>	<input>.ll	-o <path>, -O0, -O1, -O2, -O3, -g

Compiler Command	Default Output	Options
`sage --compile-llvm <input.sage>`	`<input-without-.sage>`	`-o <path>`, `-O0`, `-O1`, `-O2`, `-O3`, `-g`
`sage --emit-asm <input.sage>`	`<input>.s`	`-o <path>`, `--target <arch[-profile]>`, `-O0`, `-O1`, `-O2`, `-O3`, `-g`
`sage --compile-native <input.sage>`	hosted: `<input-without-.sage>`; non-hosted profiles: `<input-without-.sage>.o`	`-o <path>`, `--target <arch[-profile]>`, `-O0`, `-O1`, `-O2`, `-O3`, `-g`
`sage --emit-pico-c <input.sage>`	`<input>.pico.c`	`-o <path>`
`sage --compile-pico <input.sage>`	`.tmp/<program-name>` plus `<program-name>.uf2`	`-o <dir>`, `--board <name>`, `--name <program>`, `--sdk <path>`, `--chip <type>`
`--chip <type>`	`--compile-pico`	Chip type (`rp2040`, `rp2350-arm`, or `rp2350-riscv`); defaults to `rp2040`
`sage --compile-bare <input.sage>`	`<input-without-.sage>.elf`	`-o <path>`, `--target <arch>`, `-O0`, `-O1`, `-O2`, `-O3`, `-g`
`sage --compile-uefi <input.sage>`	`<input-without-.sage>.efi`	`-o <path>`, `--target x86_64|aarch64`, `-O0`, `-O1`, `-O2`, `-O3`, `-g`
`sage --jit <input.sage>`	JIT executable (temporary)	`-o <path>` for self-extracting executable with module bundling

Option	Applies To	Meaning
`-o <path>`	All emit/compile commands, including `--compile-pico`	Output file or build directory
`--cc <compiler>`	`--compile`	Overrides the host C compiler; defaults to `cc`
`--target <arch[-profile]>`	`--emit-asm`, `--compile-native`	Base targets: `x86-64`, `x86_64`, `aarch64`, `arm64`, `rv64`, `riscv64`, `mips`, `mips32`, `mips74k`; profile suffixes: `-baremetal`, `-osdev`, `-uefi`
`-O0` / `-O1` / `-O2` / `-O3`	C, LLVM, and native codegen commands	Selects the optimization pass level

Option	Applies To	Meaning
`-g`	C, LLVM, asm, and native codegen commands	Enables debug information in generated output
`--math-work=MODES`	All runtime modes	Comma-separated list of default math visualization formats: `grade`, `exec`, `bitwise`
`--board <name>`	`--compile-pico`	Pico board name; defaults to `pico`
`--name <program>`	`--compile-pico`	Overrides the generated program name derived from the input file
`--sdk <path>`	`--compile-pico`	Pico SDK path; falls back to `PICO_SDK_PATH`

LLVM backend notes:

- `--compile-llvm` produces a fully native binary by emitting LLVM IR and invoking `clang` to compile and link it. The result is a standalone executable with no Sage runtime dependency.
- As a real-world data point, the SageLLM chatbot (`models/chatbots/sagellm_chatbot.sage`) compiles to a **124 KB standalone binary** via `--compile-llvm`, including the full inference loop and tokenizer.

Profile notes for native backend:

- `hosted` (no suffix): default behavior.
- `-baremetal` / `-osdev`: emits `sage_entry` and freestanding object-oriented output.
- `-uefi`: emits `efi_main`; current stage emits a freestanding object, with full PE/COFF image linking planned as follow-up work.

6.3 Program Execution Flow

1. **Read source** from file.
2. **Initialize lexer** with source string.
3. **Initialize parser** (advance to first token).
4. **Create global environment** and initialize stdlib.
5. **Parse declarations** (statements and function definitions).
6. **Execute each statement** using the selected runtime backend.
7. **Return** at EOF.

6.4 Runtime Backends

The C-hosted `sage` binary now supports several runtime selections:

Mode	Command	Current Behavior
`ast`	`sage --runtime ast file.sage`	Original tree-walking interpreter; highest maturity and easiest to debug
`bytecode`	`sage --runtime bytecode file.sage`	Lowers each top-level statement to bytecode and executes it on the stack VM
`auto`	`sage --runtime auto file.sage`	Default; resolves to JIT on hosted, AST on bare-metal
`jit`	`sage --runtime jit file.sage`	Enables JIT profiling and type feedback

Mode	Command	Current Behavior
`aot`	`sage --runtime aot file.sage`	Enables Ahead-of-Time type specialization

Current VM architecture:

- The lexer and parser are unchanged; the VM reuses the same AST front-end.
- Each parsed top-level statement is compiled to a transient bytecode chunk, then executed immediately.
- The C-hosted toolchain can also emit a strict ahead-of-time VM artifact with `sage --emit-vm file.sage` and execute it later with `sage --run-vm file.svm`.
- The self-hosted CLI can emit the same artifact format with `sage sage.sage --emit-vm file.sage`, including compiled proc bodies and returns.
- `sage --runtime bytecode` remains **hybrid** today: unsupported top-level statements fall back to the AST interpreter through an explicit AST-bridge opcode instead of failing the whole program.
- `sage --emit-vm` is intentionally stricter: unsupported constructs fail compilation instead of silently bridging at runtime.
- Values, environments, modules, classes, instances, and the GC are shared between AST and bytecode execution.

What runs natively in the VM today:

- Literals (`number`, `string`, `bool`, `nil`)
- Global variables (`let`, assignment, reads)
- Arithmetic, comparison, logical, and bitwise operators
- Arrays, tuples, dicts, indexing, slicing, and property access
- `print`, expression statements, `if`, `while`, and array `for`
- `break` and `continue` inside while/for loops (compiled natively with loop context stack)
- Ahead-of-time proc definitions, proc calls, nested proc calls, explicit `return`, and implicit `nil` returns
- Calls to native functions, Sage functions, classes, and instance methods
- 30 GPU hot-path opcodes for frame-loop performance (poll_events, key_pressed, cmd_draw, etc.)

What still bridges or stays unsupported:

- In hybrid `--runtime bytecode` mode: class definitions, module imports, exception handling (try/catch/raise), defer, match, yield, and async procs fall back to the AST interpreter via `BC_OP_EXEC_AST_STMT`. Opcodes are defined for future native support (`BC_OP_CLASS`, `BC_OP_IMPORT`, `BC_OP_SETUP_TRY`, `BC_OP_RAISE`, etc.).
- In strict `--emit-vm` mode: these constructs fail compilation instead of bridging.
- `EXPR_AWAIT` is not supported in either mode.

Security: The VM validates all constant pool accesses (`VM_CHECK_CONST`) and AST statement indices (`VM_CHECK_AST`) to prevent buffer overflow from malformed bytecode. Stack depth is bounded at 1024 entries. All memory allocation uses OOM-safe wrappers.

The practical result is that `bytecode` mode is already useful for long-running scripts and engine-style workloads, `--emit-vm` is now a real ahead-of-time path for a meaningful strict subset of Sage, and `ast` mode remains the reference path for maximum behavioral confidence.

Part 7: Advanced Topics and Internals

7.1 Memory Leaks and GC Limitations

Known Limitations: - **String interning**: Strings are not interned; identical strings create separate allocations. - **Non-GC allocations**: AST nodes, token buffers, and other manual allocations outside the

`Value` heap still need explicit ownership and cleanup discipline. - **Manual `malloc`**: Some internal allocations (e.g., AST nodes, Token data) use `malloc` without GC tracking. Only heap-allocated Values are GC'd.

Cycle collection:

- The GC is now a tracing mark-and-sweep collector that can reclaim **circular references** among GC-managed Sage values such as arrays, dicts, tuples, classes, instances, functions, and generators.
- Cycles are only collectable when every object in the cycle is represented in the managed `Value` graph. Raw pointers, external allocations, and ad hoc native-side ownership still sit outside that guarantee.

LLVM Backend Loop Variable Limitation:

- **For-loop iteration variables cannot be modified to simulate a break in LLVM-compiled binaries.** Code like `j = len(arr)` inside a for loop has no effect on the loop control variable in `--compile-llvm` output. Always use `break` to exit loops early. This works correctly in the interpreter and all other compiler backends (C, native).

Practical Tips: - Call `gc_collect()` periodically in long-running loops to prevent heap explosion. - Avoid creating large temporary structures in loops. - Reuse arrays/dicts by modifying in-place rather than creating new ones.

7.2 Performance Characteristics

Interpreter Overhead: - **AST runtime**: Walks the parsed tree directly. It is still the simplest and most mature execution path. - **Bytecode VM**: Adds a statement-at-a-time lowering step, then executes stack bytecode. On mixed arithmetic/loop workloads it is already modestly faster than AST mode. - **Threaded Dispatch**: The VM uses computed gotos (threaded interpretation) for high-performance instruction dispatch on supported compilers, significantly reducing loop overhead. - **Register Spilling**: The VM stack pointer and instruction pointer are kept in CPU registers where possible, minimizing memory traffic during the execution loop. - **Nested scopes**: Linear search up parent chain; O(depth) for variable lookup. - **GC pauses**: Mark-and-sweep still pauses execution during collection, but the trigger is now dynamic and based on both object count and managed bytes.

Optimization Opportunities: - Bytecode functions and method bodies, so fewer calls bridge back to AST. - Local-slot or register-based VM frames instead of env-backed global lookups for more code. - Variable lookup caching (memoization). - Generational GC (mark only young objects frequently). - JIT compilation for hot paths.

3 Current Recipe Benchmark

The repository now includes a five-recipe benchmark:

```
python3 scripts/benchmark_recipes.py --runs 5 --warmups 1
```

It measures:

- `sage-interpreted-c-ast`: the C-hosted tree-walking interpreter
- `sage-interpreted-vm`: the C-hosted bytecode VM runtime
- `sage-compiled-c`: the native C backend compiled to a host executable
- `sage-compiled-vm`: the C-hosted ahead-of-time VM artifact (`--emit-vm` + `--run-vm`)
- `sage-compiled-sage`: the self-hosted Sage compiler path that emits C and then compiles it with the host toolchain

Workload source: `benchmarks/runtime_compare.sage`

Generated chart assets:

- `assets/charts/benchmark-recipes-total.svg`
- `assets/charts/benchmark-recipes-run.svg`

Current caveats:

- `sage-compiled-vm` now passes the default benchmark workload and is charted as a real execution lane, but it is still a strict subset backend: unsupported constructs fail during `--emit-vm` instead of falling back at runtime.
- `sage-compiled-sage` is still experimental and currently fails checksum validation on the default benchmark workload, so it is called out in the chart footer instead of being charted as a valid timing result.

Interpretation:

- `sage-interpreted-vm` is the direct runtime-backend comparison against `sage-interpreted-c-ast`.
- `sage-compiled-c` has the lowest execution-only runtime on the default workload, but its total wall time includes code generation and host compilation.
- The total-time chart answers "time to result"; the execution-only chart answers "steady-state runtime after the binary already exists."

3 Sage vs Python 3 Benchmarks

A separate benchmark suite compares all Sage execution paths against CPython 3.x:

```
make benchmark-python        # Console table output
make benchmark-python-md     # Markdown table output
```

10 paired workloads (`benchmarks/01_fibonacci.sage` + `.py` through `benchmarks/10_primes_sieve`):

Benchmark	What it tests
fibonacci	Recursive function call overhead
loop_sum	Raw loop + arithmetic throughput
string_concat	String allocation + GC pressure
array_ops	Dynamic array growth + iteration
dict_ops	Hash table insert + lookup
class_method	OOP method dispatch
nested_loops	Control flow with break/continue
exception_handling	Try/catch overhead
recursion_closures	Closure creation + higher-order calls
primes_sieve	Array access + conditional logic

Five recipes tested: Python 3, Sage AST, Sage VM, Sage C (compiled), Sage LLVM (compiled).

Typical performance characteristics: - **LLVM compiled**: 2-8x faster than CPython (fibonacci 3.9x, loop sum 7.7x) - **C compiled**: 2-4x faster than CPython on most workloads - **AST interpreter**: On par with CPython (faster on string/array ops, slower on deep recursion) - **Bytecode VM**: Similar to AST, with measurable speedups on tight loops

7.3 Extending SageLang with Native Functions

Adding a Native Function:

```
// In interpreter.c, add:
static Value my_function(int argCount, Value* args) {
  if (argCount != 2) return val_nil();
  if (!IS_NUMBER(args[0]) || !IS_NUMBER(args[1])) return val_nil();
  double result = AS_NUMBER(args[0]) + AS_NUMBER(args[1]) * 2;
  return val_number(result);
}

// In init_stdlib(), add:
env_define(env, "my_func", 7, val_native(my_function));
```

Usage from SageLang:

```
print my_func(3, 4)  # 3 + 4*2 = 11
```

7.4 Debugging and Introspection

Print Value:

```
void print_value(Value v);  // In value.c
```

GC Stats:

```
let stats = gc_stats()
print stats  # {"bytes_allocated": ..., "num_objects": ..., ...}
```

Tracing Execution (manual): - Add `printf()` statements to interpreter.c before/after statement/expression evaluation. - Mark garbage objects: Modify `gc_sweep()` to not free unmarked objects, inspect them.

7.5 Interpreter Safety Limits

SageLang enforces several compile-time and runtime limits to prevent crashes from malicious or malformed input:

Limit	Constant	Value	Location	Behavior on Violation
Recursion depth (logical)	`MAX_RECURSION_DEPTH`	12000	interpreter.c	Catchable exception
Recursion depth (real stack)	`stack_danger()`	75% of `RLIMIT_STACK`	interpreter.c	Catchable exception — fires before the OS stack is exhausted regardless of frame size
Parser nesting	`MAX_PARSER_DEPTH`	100000	parser.c	Parse error
Loop iterations	`MAX_LOOP_ITERATIONS`	1,000,000	interpreter.c	Catchable exception
String literal length	`MAX_STRING_LENGTH`	4096	lexer.c	Parse error
Function arguments	Stack array	255	interpreter.c	Runtime error

The stack-proximity guard (v4.1.16) tracks a per-thread stack origin and refuses recursion once within the safety margin of the real OS limit, so deep or fat-framed call chains fail with a catchable exception instead of SIGSEGV. At startup the runtime also raises the soft stack limit to 512 MB when the hard limit permits, giving self-hosted double-interpretation chains headroom.

Null function guards: The interpreter checks for null pointers in both `VAL_FUNCTION` and `VAL_NATIVE` call paths before dispatch. A null callee produces a runtime error and returns `nil`.

Type-safe accessor macros (`value.h`):

Macro	Returns on type mismatch
`SAGE_AS_STRING(v)`	`""` (empty string)
`SAGE_AS_NUMBER(v)`	`0.0`
`SAGE_AS_BOOL(v)`	`0` (false)

These are intended for native function implementations that want defensive coercion. The standard `IS_*/AS_*` pattern (check type, then access) remains the recommended approach for most code:

```
static Value my_native(int argCount, Value* args) {
    if (argCount < 1 || !IS_STRING(args[0])) return val_nil();
    const char* s = AS_STRING(args[0]);  // safe - guarded by IS_STRING above
    // ...
}
```

Part 8: Future Directions and Design Notes

8.1 Planned Features (Incomplete Implementations)

Pattern Matching (`match`/`case`): Fully implemented across all backends (see Section 4.9).

Defer Statements: Fully implemented with LIFO ordering and scope-exit semantics (see Section 4.8).

Multiple Inheritance / Traits: - Currently single-parent inheritance only.

Operator Overloading: - No support for custom `__add__()`, etc.

Type Annotations: - Fully supported for variables (`let x: Int = 42`) and procedures (`proc add(a: Int, b: Int) -> Int:`). - Dotted names are supported for qualified types (e.g., `let x: vfs.VFS`). - Type checking is enforced during a static analysis pass (`pass_typecheck` in `typecheck.c`) prior to compilation or interpretation.

8.2 Design Decisions and Rationale

Why Indentation-Based Syntax? - Python familiarity for embedded systems developers. - Reduces bracket clutter in embedded/IoT code. - Lexer complexity worth the syntactic clarity.

Why Mark-and-Sweep Over Reference Counting? - Simpler to implement; no cycle detection overhead. - Acceptable for embedded contexts where pause is tolerable. - Future: Generational GC for better pause times.

Why Tree-Walking Interpreter? - Rapid prototyping and debugging. - Minimal complexity; easy to extend. - Suitable for small programs; not for performance-critical workloads.

Why Classes Over Functions-as-Objects? - Explicit OOP model matches C++ style (familiar to embedded developers). - Clearer semantics for `self` and method dispatch.

Part 9: Bundled Library Modules (lib/)

SageLang ships with a set of **pure-Sage library modules** in the `lib/` directory. These are importable via `import` or `from ... import` when running from the project root (the module resolver searches `./lib/`).

> **Note**: `lib/math.sage` and native `math` share a name — the native module takes precedence. Use `import math` for native math functions (sin, cos, sqrt, etc.) and `from math import ...` for native math. The lib/math.sage helpers (factorial, gcd, etc.) are available if no native module shadows them.

9.1 Arrays Module (`lib/arrays.sage`)

Functional array utilities:

```
from arrays import map, filter, reduce, reverse, unique, flatten, zip, chunk
```

```
proc double(x):
    return x * 2

print map([1, 2, 3], double)       # [2, 4, 6]
print filter([1, 2, 3, 4], is_even) # [2, 4]
print reverse([1, 2, 3])            # [3, 2, 1]
print unique([1, 2, 2, 3])          # [1, 2, 3]
print flatten([[1, 2], [3, 4]])      # [1, 2, 3, 4]
print zip([1, 2], ["a", "b"])        # [(1, a), (2, b)]
print chunk([1, 2, 3, 4, 5], 2)     # [[1, 2], [3, 4], [5]]
```

Function	Description
`copy(arr)`	Shallow copy
`append_all(target, extra)`	Extend target in-place
`concat(a, b)`	Return new merged array
`reverse(arr)`	Return reversed copy
`map(arr, fn)`	Apply fn to each element
`filter(arr, pred)`	Keep elements where pred is true
`reduce(arr, init, fn)`	Fold left with accumulator
`contains(arr, val)`	Check membership
`index_of(arr, val)`	Find index (-1 if missing)
`find(arr, pred)`	First element matching predicate
`unique(arr)`	Remove duplicates
`flatten(nested)`	Flatten one level of nesting
`take(arr, n)`	First n elements
`drop(arr, n)`	Skip first n elements
`zip(a, b)`	Pair elements as tuples
`chunk(arr, size)`	Split into chunks

9.2 Strings Module (`lib/strings.sage`)

String manipulation utilities:

```
from strings import words, compact, pad_left, dash_case, from_bin

print words("  hello   world  ")  # [hello, world]
print compact("  hello   world  ") # hello world
print pad_left("42", 5, "0")       # 00042
print dash_case("hello world")     # hello-world
print from_bin("0b1010")           # 10
```

Function	Description
`words(text)`	Split on whitespace, remove empties
`compact(text)`	Collapse whitespace to single spaces
`contains(text, sub)`	Substring check
`count_substring(text, sub)`	Count occurrences
`repeat(text, n)`	Repeat string n times
`pad_left(text, width, pad)`	Left-pad to width
`pad_right(text, width, pad)`	Right-pad to width
`surround(text, left, right)`	Wrap with delimiters
`csv(values)`	Join with commas
`dash_case(text)`	Convert to dash-case
`snake_case(text)`	Convert to snake_case

Function	Description
`endswith(text, suffix)`	Check suffix
`from_bin(bits)`	Binary string to number

9.3 Dicts Module (`lib/dicts.sage`)

Dictionary query and manipulation helpers:

```
from dicts import has, size, get_or, has_all, entries, remove_keys

let d = {}
d["x"] = 10
d["y"] = 20

print has(d, "x")                    # true
print size(d)                        # 2
print get_or(d, "z", "default")      # default
print has_all(d, ["x", "y"])         # true
```

Function	Description
`keys(d)`	Wrapper for `dict_keys`
`values(d)`	Wrapper for `dict_values`
`size(d)`	Number of keys
`has(d, key)`	Key existence check
`get_or(d, key, fallback)`	Get with default
`entries(d)`	Array of `(key, value)` tuples
`has_all(d, keys)`	All keys present?
`has_any(d, keys)`	Any key present?
`select_values(d, keys, fallback)`	Get multiple values
`remove_keys(d, keys)`	Delete keys in-place
`count_missing(d, keys)`	Count absent keys

9.4 Iter Module (`lib/iter.sage`)

Reusable generator functions:

```
from iter import count, range_step, cycle, enumerate_array, take

let evens = range_step(0, 20, 2)
print take(evens, 5)  # [0, 2, 4, 6, 8]

let c = cycle([1, 2, 3])
print take(c, 7)  # [1, 2, 3, 1, 2, 3, 1]

let e = enumerate_array(["a", "b", "c"])
print next(e)  # (0, a)
print next(e)  # (1, b)
```

Function	Description
`count(start, step)`	Infinite counter generator
`range_step(start, end, step)`	Range with custom step
`repeat(value, count)`	Yield value n times

Function	Description
`repeat_forever(value)`	Infinite repeat
`enumerate_array(arr)`	Yield `(index, value)` tuples
`cycle(arr)`	Infinite cycle through array
`take(gen, n)`	Collect n values from generator
`nth(gen, index)`	Get nth value from generator

9.5 Stats Module (`lib/stats.sage`)

Statistical functions:

```
from stats import mean, variance, stddev, cumulative, normalize

print mean([1, 2, 3, 4, 5])       # 3
print variance([1, 2, 3, 4, 5])   # 2
print cumulative([1, 2, 3, 4, 5]) # [1, 3, 6, 10, 15]
print normalize([1, 2, 3, 4, 5])  # [0, 0.25, 0.5, 0.75, 1]
```

Function	Description
`sum(values)`	Sum of array
`product(values)`	Product of array
`min_value(values)`	Minimum element
`max_value(values)`	Maximum element
`mean(values)`	Arithmetic mean
`range_span(values)`	max - min
`cumulative(values)`	Running totals
`variance(values)`	Population variance
`stddev(values)`	Standard deviation
`normalize(values)`	Scale to [0, 1]

9.6 Utils Module (`lib/utils.sage`)

General-purpose helpers:

```
from utils import default_if_nil, choose, between, head, last, repeat_value

print default_if_nil(nil, 42)   # 42
print choose(true, "yes", "no") # yes
print between(5, 1, 10)         # true
print head([1, 2, 3])           # 1
print last([1, 2, 3])           # 3
print repeat_value(0, 5)        # [0, 0, 0, 0, 0]
```

Function	Description
`identity(x)`	Return x unchanged
`choose(cond, a, b)`	Ternary selection
`default_if_nil(val, fallback)`	Nil coalescing
`is_even(n)` / `is_odd(n)`	Parity check
`between(val, lo, hi)`	Range check
`swap(a, b)`	Return `(b, a)` tuple
`head(arr)` / `last(arr)`	First/last element (nil if empty)
`repeat_value(val, n)`	Array of n copies

Function	Description
`times(n, fn)`	Call fn(i) n times

9.7 Assert Module (`lib/assert.sage`)

Test assertion helpers (raise on failure):

```
from assert import assert_equal, assert_true, assert_close

assert_true(len([1, 2, 3]) == 3, "wrong length")
assert_equal(2 + 2, 4, "math is broken")
assert_close(3.14, 3.14159, 0.01, "not close enough")
```

Function	Description
`fail(msg)`	Always raise
`assert_true(cond, msg)`	Raise if false
`assert_false(cond, msg)`	Raise if true
`assert_equal(actual, expected, msg)`	Raise if not equal
`assert_nil(val, msg)`	Raise if not nil
`assert_not_nil(val, msg)`	Raise if nil
`assert_close(actual, expected, tol, msg)`	Raise if difference > tolerance
`assert_array_contains(arr, val, msg)`	Raise if val not in arr

9.8 Math Module (`lib/math.sage`)

Pure-Sage math helpers (shadowed by native `math` module — use when native module is unavailable):

Function	Description
`add`, `sub`, `mul`, `div`	Basic arithmetic
`min`, `max`, `abs`, `sign`	Comparison helpers
`clamp(val, lo, hi)`	Clamp to range
`square(x)`, `cube(x)`	Powers
`lerp(a, b, t)`	Linear interpolation
`pow_int(base, exp)`	Integer exponentiation
`factorial(n)`	n!
`gcd(a, b)`, `lcm(a, b)`	GCD and LCM
`sum(arr)`, `product(arr)`, `mean(arr)`	Aggregates
`sqrt(n)`	Newton's method approximation
`printm(expr, backend, formats)`	Evaluate expression and show work
`distance(x1, y1, x2, y2)`	Euclidean distance

Arithmetic Visualization (`printm`):

The `math.printm()` function evaluates an arithmetic expression provided as a string and visualizes the step-by-step "work" for each operation.

```
import math
math.printm("123 + 456 * 2", backend="sage")
```

- `backend`:
  - `"sage"` (default): Pure SageLang implementation using vertical "grade-school" format.

  - `"c"`: Uses native C-level arithmetic bridges.
  - `"asm"`: Uses inline assembly (x86-64, aarch64, rv64, mips) and calls `printf` via FFI to show the instruction-level execution.
- `formats`: An array of strings to customize output.
  - `"grade"`: Vertical grade-school arithmetic visualization.
  - `"exec"`: Show the underlying execution mode (e.g., "[ASM] Using ADD instruction").
  - `"bitwise"`: Reserved for bitwise operation visualization.

The default format can be configured globally using the `--math-work` CLI flag. | `normalize(val, lo, hi)` | Scale to [0, 1] |

9.9 GPU Libraries

SageLang ships with 18 GPU/rendering library modules in `lib/graphics/`. All are imported with the `graphics.` prefix (e.g., `import graphics.vulkan` binds as `vulkan`):

Module	Import	Lines	Purpose
`vulkan.sage`	`import graphics.vulkan`	~425	Ergonomic Vulkan builder API (string-based buffer/shader/pipeline creation)
`gpu.sage`	`import graphics.gpu`	~150	High-level compute helpers (one-shot dispatch, ping-pong buffers)
`opengl.sage`	`import graphics.opengl`	~400	Drop-in OpenGL backend (same API as gpu module, OpenGL 4.5 init)
`ui.sage`	`import graphics.ui`	~400	Immediate-mode GUI widgets (windows, buttons, sliders, menus, text inputs)
`math3d.sage`	`import graphics.math3d`	~250	vec2/3/4, mat4, perspective/ortho projections, camera utilities
`mesh.sage`	`import graphics.mesh`	~300	Procedural geometry (cube, plane, sphere), OBJ loading, GPU upload
`renderer.sage`	`import graphics.renderer`	~200	Frame loop management (depth buffer, render pass, sync)
`material.sage`	`import graphics.material`	~150	Shader+texture+descriptor binding, material presets
`scene.sage`	`import graphics.scene`	~200	Node hierarchy, transforms, traversal, find_by_name
`pbr.sage`	`import graphics.pbr`	~300	Cook-Torrance PBR materials, point/directional lights, IBL
`postprocess.sage`	`import graphics.postprocess`	~250	HDR targets, bloom chain, tone mapping (ACES/Reinhard)
`shadows.sage`	`import graphics.shadows`	~200	Shadow maps, depth-only passes, cascade shadow maps

Module	Import	Lines	Purpose
deferred.sage	import graphics.deferred	~300	G-buffer (4 MRT), SSAO (32 samples), SSR (64-step raymarch)
taa.sage	import graphics.taa	~150	Temporal anti-aliasing (Halton jitter, history blend)
gltf.sage	import graphics.gltf	~200	glTF 2.0 JSON loading, mesh/material extraction
asset_cache.sage	import graphics.asset_cache	~100	Shader/texture/mesh caching and deduplication
frame_graph.sage	import graphics.frame_graph	~150	Pass dependency ordering (topological sort)
debug_ui.sage	import graphics.debug_ui	~150	FPS tracking, custom debug values, toggle overlay

9.10 OS Development Libraries

SageLang ships with 52 OS/bare-metal development modules across `lib/os/`, `lib/os/boot/`, `lib/os/kernel/`, and `lib/os/image/`:

Module	Import	Purpose
fat.sage	import os.fat	FAT8/12/16/32 boot sector parser, cluster-to-LBA, FAT entry offsets
fat_dir.sage	import os.fat_dir	FAT directory traversal, file reading, path resolution, cluster chains
elf.sage	import os.elf	ELF32/64 header, program/section headers, string table lookup
mbr.sage	import os.mbr	MBR partition table, CHS decode, bootable partition finder
gpt.sage	import os.gpt	GPT header, GUID parsing, partition type identification
pe.sage	import os.pe	PE/COFF binary parser, DOS/COFF/optional headers, UEFI app detection
pci.sage	import os.pci	PCI config space (Type 0/1), BAR decode, capability lists
uefi.sage	import os.uefi	EFI memory map, config tables, RSDP, ACPI SDT headers
acpi.sage	import os.acpi	MADT (APIC), FADT, HPET, MCFG parsers, processor enumeration
paging.sage	import os.paging	x86-64 page tables, PTE flags, identity/higher-half mapping helpers
idt.sage	import os.idt	x86-64 IDT gate construction, exception vectors, PIC remapping
serial.sage	import os.serial	UART/COM port configuration, init sequences, debug output encoding
dtb.sage	import os.dtb	Flattened Device Tree parser for ARM64/RISC-V (nodes, properties, search)

Module	Import	Purpose
`alloc.sage`	`import os.alloc`	Bump, free-list, and bitmap page allocators for kernel memory
`vfs.sage`	`import os.vfs`	Virtual filesystem layer with mount table, path utilities, memfs backend

Module	Import	Purpose
ext.sage" |import os.ext| ext2/3/4 superblock, inode table, directory entries, extent tree |	Linux /sys filesystem interface	
|btrfs.sage|import os.btrfs| Btrfs superblock, chunk tree, root tree, subvolumes, checksums |		
|f2fs.sage|import os.f2fs| F2FS superblock, checkpoint, segment info, node/data addressing |		
|boot/multiboot.sage|import os.boot.multiboot| Multiboot2 header generation, tag building, boot info parsing |		
|boot/gdt.sage|import os.boot.gdt| x86_64 GDT descriptor construction, TSS entries, LGDT sequence |		
|boot/start.sage|import os.boot.start| x86_64 startup assembly generation (long mode entry, stack setup) |		
|boot/linker.sage|import os.boot.linker| Linker script generation for bare-metal ELF kernels |		
|boot/bios.sage|import os.boot.bios| BIOS interrupt interface (INT 0x10, 0x13, 0x15, 0x16) for legacy x86 boot |		
|boot/e820.sage|import os.boot.e820| BIOS memory map collection and normalization |		
|boot/a20.sage|import os.boot.a20| A20 gate enabling methods (BIOS, Fast A20, Keyboard Controller) |		
|boot/cpuid.sage|import os.boot.cpuid| CPU feature detection (Long mode, MSR, SSE, Vendor/Brand strings) |		
|boot/real_mode.sage|import os.boot.real_mode| Helpers for segment:offset addressing and real-mode stack setup |		
|boot/prot_mode.sage|import os.boot.prot_mode| Transition logic from real mode to 32-bit protected mode |		
|boot/long_mode.sage|import os.boot.long_mode| Transition logic from		

9.11 Networking Libraries

SageLang ships with 8 high-level networking modules in `lib/net/`, built on top of the native `socket`, `tcp`, `http`, and `ssl` modules:

Module	Import	Purpose
`url.sage`	`import net.url`	URL parsing/building, percent-encoding/decoding, query strings
`headers.sage`	`import net.headers`	HTTP header parsing/building, content-type inspection, constants
`request.sage`	`import net.request`	HTTP request builder with fluent API, auth helpers, status utilities
`server.sage`	`import net.server`	TCP/HTTP server framework with routing, request parsing, response builders
`websocket.sage`	`import net.websocket`	WebSocket frame building/parsing (RFC 6455), upgrade handshake
`mime.sage`	`import net.mime`	MIME type lookup from file extensions (80+ types)
`dns.sage`	`import net.dns`	DNS wire-format message parsing/building, name compression
`ip.sage`	`import net.ip`	IPv4 parsing/validation, CIDR subnets, private/loopback/multicast checks

9.12 Cryptography Libraries

SageLang ships with 6 cryptography modules in `lib/crypto/`:

Module	Import	Purpose
`hash.sage`	`import crypto.hash`	SHA-256, SHA-1, CRC-32 hash functions with hex output
`hmac.sage`	`import crypto.hmac`	HMAC (RFC 2104) with pluggable hash, constant-time comparison
`encoding.sage`	`import crypto.encoding`	Base64 (standard + URL-safe), hex encoding/decoding
`cipher.sage`	`import crypto.cipher`	XOR cipher, RC4 stream cipher, PKCS#7 padding, CBC/CTR mode helpers
`rand.sage`	`import crypto.rand`	xoshiro256** PRNG, UUID v4, random bytes/strings/hex, shuffle
`password.sage`	`import crypto.password`	PBKDF2-HMAC key derivation, password hashing/verification

9.13 Machine Learning Libraries

SageLang ships with 10 PyTorch-style machine learning modules in `lib/ml/`:

Module	Import	Purpose
`tensor.sage`	`import ml.tensor`	N-dimensional tensors, element-wise ops, matmul, reductions, activations
`nn.sage`	`import ml.nn`	Neural network layers (Linear, ReLU, Sigmoid, Dropout), Sequential model
`optim.sage`	`import ml.optim`	SGD (momentum), Adam optimizer, learning rate schedulers
`loss.sage`	`import ml.loss`	MSE, cross-entropy, Huber, L1, hinge, KL divergence
`data.sage`	`import ml.data`	Dataset/DataLoader, batching, normalization, train/test split
`debug.sage`	`import ml.debug`	Weight stats, histograms, activation analysis, gradient checking, training diagnostics
`viz.sage`	`import ml.viz`	SVG chart generation (loss curves, weight distributions, attention heatmaps, architecture diagrams)
`monitor.sage`	`import ml.monitor`	Live training monitor, progress bars, memory snapshots, throughput, checkpoints
`gpu_accel.sage`	`import ml.gpu_accel`	GPU offload for tensor ops and training; bridges ML workloads to the Vulkan/OpenGL GPU backend
`npu.sage`	`import ml.npu`	NPU backend for Qualcomm Hexagon (SNPE), Samsung Exynos (ONE), NNAPI, and ARM NEON SIMD fallback

Native ML Backend (`ml_native`) `ml_native` is a built-in native C module (not a `lib/ml/` Sage file) that exposes 24 high-performance functions directly from `src/c/ml_backend.c`:

`matmul`, `add`, `scale`, `relu`, `gelu`, `silu`, `sigmoid`, `softmax`, `layer_norm`, `rms_norm`, `cross_entropy`, `adam_update`, `clip_grad`, `benchmark`, `train_step`, `forward_pass`, `load_weights`, `gpu_available`, `set_gpu_threshold`, `cpu_count`, `auto_parallel`, `set_threads`, `set_parallel_threshold`, `get_threads`

Backpropagation and training:

- `ml_native.train_step()` — C-level combined forward pass + backward pass + SGD weight update for transformer training. Avoids Python/Sage overhead in the hot loop.
- `ml_native.forward_pass()` — Runs inference using the same computation graph as `train_step`, ensuring numerical consistency between training and evaluation.
- `ml_native.load_weights(path)` — Loads trained weights from a binary file via native C parser; no Sage I/O overhead.

Standalone C trainer: `make train-c` builds a standalone C training binary (no Sage runtime required) that uses `ml_backend.c` directly. Auto-detects cuBLAS GPU acceleration and ARM NEON SIMD; also works on mobile via Termux + proot ARM64. Additional targets: `make train-sage` (Sage interpreter), `make chatbot-c` / `make chatbot-llvm` (compile chatbots), `make sl-tq-chat` (SL-TQ-LLM generative chatbot).

9.14 CUDA Libraries

SageLang ships with 4 CUDA abstraction modules in `lib/cuda/`:

Module	Import	Purpose
`device.sage`	`import cuda.device`	GPU device descriptors, compute capability, feature detection
`memory.sage`	`import cuda.memory`	GPU memory allocation, typed tensors, memory pools
`kernel.sage`	`import cuda.kernel`	Kernel definition, launch parameters, occupancy analysis
`stream.sage`	`import cuda.stream`	CUDA streams, events, multi-stream execution plans

9.15 Standard Library (`lib/std/`)

SageLang ships with 23 general-purpose standard library modules in `lib/std/`:

Core utilities:

Module	Import	Purpose
`regex.sage`	`import std.regex`	Regular expression engine (., *, +, ?, [], \\d, \\w, \\s)
`datetime.sage`	`import std.datetime`	Date/time creation, ISO 8601, arithmetic, comparison
`log.sage`	`import std.log`	Structured logging (TRACE-FATAL), handlers, child loggers
`argparse.sage`	`import std.argparse`	CLI argument parser (flags, options, positionals)
`compress.sage`	`import std.compress`	RLE, LZ77, delta encoding/decoding
`process.sage`	`import std.process`	Environment, path manipulation, exit codes, timers
`unicode.sage`	`import std.unicode`	UTF-8, character classification, case conversion, trim
`fmt.sage`	`import std.fmt`	Number/string formatting, templates, table output
`testing.sage`	`import std.testing`	Test runner, assertions, benchmarks
`db.sage`	`import std.db`	In-memory DB (CRUD, joins, aggregation, pagination)
`signal.sage`	`import std.signal`	Event bus (pub/sub), atexit handlers

Type system extensions:

Module	Import	Purpose
`enum.sage`	`import std.enum`	Enums, tagged unions, Result (Ok/Err), Option (Some/None)
`trait.sage`	`import std.trait`	Interface/trait system, behavioral contracts

Concurrency primitives:

Module	Import	Purpose
`channel.sage`	`import std.channel`	Go-style channels, send/recv/select, fan-in/out
`threadpool.sage`	`import std.threadpool`	Work queue, parallel map, futures/promises
`atomic.sage`	`import std.atomic`	Atomic integers, CAS, spin locks, counters
`rwlock.sage`	`import std.rwlock`	Read-write locks, scoped lock helpers
`condvar.sage`	`import std.condvar`	Condition variables, barriers, latches, semaphores

Developer tooling:

Module	Import	Purpose
`debug.sage`	`import std.debug`	Value inspection, trace logging, watch expressions
`profiler.sage`	`import std.profiler`	Hierarchical timing, hotspots, benchmark runner
`docgen.sage`	`import std.docgen`	Doc extraction from comments, markdown generation
`build.sage`	`import std.build`	Project config, dependencies, semver, build targets
`interop.sage`	`import std.interop`	FFI helpers, C types, pack/unpack, platform detection

9.16 LLM / Neural Network Libraries

SageLang ships with 19 LLM/neural network modules in `lib/llm/` for building and training language models:

> **Backpropagation note**: For performance-critical training loops, use `ml_native.train_step()` (C-level forward+backward+SGD), `ml_native.forward_pass()` (inference, same computation graph), and `ml_native.load_weights(path)` (native weight loading) instead of the pure-Sage `llm.train` module. See Section 9.13 Native ML Backend.

Module	Import	Purpose
`config.sage`	`import llm.config`	Model configs (tiny to Llama-13B), param counting, memory estimation
`tokenizer.sage`	`import llm.tokenizer`	Character, word, and BPE tokenizers
`embedding.sage`	`import llm.embedding`	Token embeddings, sinusoidal/learned/RoPE positional encodings
`attention.sage`	`import llm.attention`	Multi-head self-attention, KV cache
`transformer.sage`	`import llm.transformer`	Transformer blocks, LayerNorm/RMSNorm, FFN, model assembly
`generate.sage`	`import llm.generate`	Text generation (greedy, top-k, top-p, beam search)
`train.sage`	`import llm.train`	Training loops, LR schedules, cross-entropy, perplexity

Module	Import	Purpose
agent.sage	import llm.agent	Agentic framework (tools, CoT, memory, planning, multi-agent)
prompt.sage	import llm.prompt	Chat formatting (ChatML/Llama/Alpaca), templates, few-shot
lora.sage	import llm.lora	LoRA fine-tuning adapters, merge-back
quantize.sage	import llm.quantize	Int8/int4 quantization, error analysis
engram.sage	import llm.engram	Persistent neural memory (working/episodic/semantic/procedural)
rag.sage	import llm.rag	Document chunking, keyword retrieval, context assembly, summarization
dpo.sage	import llm.dpo	Direct Preference Optimization, ORPO, preference pairs, reward models
gguf.sage	import llm.gguf	GGUF v3 export for Ollama/llama.cpp, Modelfile gen, quantization
gguf_import.sage	import llm.gguf_import	Import GGUF models from Ollama into Sage; converts weights to native tensor format
turboquant.sage	import llm.turboquant	TurboQuant near-optimal vector quantization (ICLR 2026): two-stage PolarQuant (random rotation + MSE-optimal scalar quantization) + QJL (1-bit residual correction); KV cache compression at 3-bit with 6x memory reduction
autoresearch.sage	import llm.autoresearch	Karpathy-style autonomous research agent; ratchet loop (propose → train → evaluate → accept/reject); built-in scale/choice/perturb strategies; research journal; multi-agent session merging
evolve.sage	import llm.evolve	Self-evolving neural architecture; progressive growth from seed (64d/1L/98K params) to ancient (512d/8L/67M params); auto-plateau detection; weight padding for width growth; identity-init for depth growth

9.17 Agent Framework (lib/agent/)

Module	Import	Purpose
core.sage	import agent.core	ReAct agent loop, tool dispatch, scratchpad, prompt building
tools.sage	import agent.tools	Pre-built tools (file I/O, code analysis, search, system)

Module	Import	Purpose
planner.sage	import agent.planner	Task decomposition with dependency DAG, auto-execution
router.sage	import agent.router	Multi-agent orchestrator, capability routing, pipelines
supervisor.sage	import agent.supervisor	Supervisor-Worker control plane, workflow engine, retries
critic.sage	import agent.critic	Verification loops, rule/LLM validators, composite checks
schema.sage	import agent.schema	Typed tool interfaces, parameter validation, bounded execution
trace.sage	import agent.trace	SFT trace recording, training data generation
grammar.sage	import agent.grammar	Grammar-constrained decoding, tool call/JSON/Sage validation
sandbox.sage	import agent.sandbox	Program-aided reasoning, code execution, math eval
tot.sage	import agent.tot	Tree of Thoughts, BFS/best-first search, state rollbacks
semantic_router.sage	import agent.semantic_router	Fast command dispatch, keyword routing, LLM bypass

9.18 Chatbot Framework (lib/chat/)

Module	Import	Purpose
bot.sage	import chat.bot	Conversation management, intents, middleware, LLM responses
session.sage	import chat.session	Multi-session store, history, export (text/JSON)
persona.sage	import chat.persona	Pre-built personas (SageDev, Teacher, Debugger, etc.)

9.19 UI Widget Library (lib/graphics/ui.sage)

Immediate-mode GPU UI system for building application interfaces:

```
import gpu
import graphics.ui

gpu.init_windowed("App", 800, 600, "My App", false)
let ctx = ui.ui_create()

while not gpu.window_should_close():
    gpu.poll_events()
    ui.ui_begin_frame(ctx)

    ui.ui_panel(ctx, 10, 10, 300, 400, "Settings")
    if ui.ui_button(ctx, 20, 50, 120, 30, "Apply"):
        print "Applied!"

    let volume = ui.ui_slider(ctx, 20, 100, 260, "Volume", 0.75)
    let dark = ui.ui_checkbox(ctx, 20, 140, "Dark Mode", true)
```

```
ui.ui_end_frame(ctx)
```

Available widgets:

Widget	Function	Returns
Label	`ui_label(ctx, x, y, text)`	nil
Button	`ui_button(ctx, x, y, w, h, label)`	bool (clicked)
Panel	`ui_panel(ctx, x, y, w, h, title)`	nil
Window	`ui_window(ctx, x, y, w, h, title)`	dict (content area)
Checkbox	`ui_checkbox(ctx, x, y, label, checked)`	bool (new state)
Slider	`ui_slider(ctx, x, y, w, label, value)`	number (0.0-1.0)
Scrollbar	`ui_scrollbar_v(ctx, x, y, h, content_h, scroll)`	number (0.0-1.0)
Menu	`ui_menu_button(ctx, x, y, w, h, label, items)`	int (item index or -1)
Text Input	`ui_text_input(ctx, x, y, w, label, text)`	string
Progress	`ui_progress(ctx, x, y, w, h, value, label)`	nil
Separator	`ui_separator(ctx, x, y, w)`	nil
Tooltip	`ui_tooltip(ctx, text)`	nil

Theming: `ctx["theme"]` is a dict with 20+ configurable colors (`bg`, `accent`, `text`, `border`, etc.) and sizes (`padding`, `font_size`, `title_height`, etc.). Call `ui.ui_default_theme()` for defaults.

9.20 AVR Assembler & Arduino Uno Support (`core/boards/AVR/`)

SageLang v4.1.7 introduces a complete two-pass assembler package targeting the **AVR 8-bit RISC architecture (ATmega328P / ATmega328PB)** such as the Arduino Uno R3.

This includes: - **`avr_assembler`**: Parses AVR assembly, handles label resolution, relative offsets for branching (`brne`, `rjmp`, `rcall`), and resolves preprocessor directives (`.org`, `.equ`, `.byte`). - **`avr_opcodes`**: Encoder translating mnemonics to exact 16-bit instructions matching the Microchip datasheet. - **`avr_hex`**: Intel HEX (I8HEX) emitter generating `.hex` files ready for `avrdude` flashing.

Standard I/O registers resolved automatically:

- `DDRB`, `PORTB`, `PINB`, `DDRC`, `PORTC`, `PINC`, `DDRD`, `PORTD`, `PIND`, `SPL`, `SPH`, `SREG`

Example Usage:

```
import avr_assembler
import avr_hex
import io

let src = "
    .org 0x0000
    ldi r16, 0x20
```

```
    out DDRB, r16
    out PORTB, r16
    ret
"
let words = avr_assembler.assemble(src)
let hex_txt = avr_hex.emit_hex(words, 0)
print(hex_txt)
```

Part 13: Self-Hosting (Phase 13)

SageLang is **self-hosted**: the lexer, parser, and interpreter have been ported from C to Sage itself. The self-hosted pipeline can execute `.sage` programs using only the C interpreter as a bootstrap host.

13.1 Architecture

The self-hosted interpreter lives in `src/sage/` and consists of:

File	Description	Size
`token.sage`	Token type constants	~50 lines
`lexer.sage`	Tokenizer with indentation tracking	~300 lines
`ast.sage`	AST node constructors (dict-based)	~100 lines
`parser.sage`	Recursive descent parser	~700 lines
`interpreter.sage`	Tree-walking evaluator with module imports	~1050 lines
`errors.sage`	Rich error reporting (Rust/Elm-style)	~200 lines
`environment.sage`	Dict-based scope/environment	~35 lines
`sage.sage`	Full CLI entry point	~200 lines

13.2 Running Self-Hosted Code

```
cd src/sage
../../sage sage.sage program.sage
```

The bootstrap reads a `.sage` file, tokenizes it, parses it to an AST, and evaluates it — all in Sage running on the C interpreter.

13.3 Key Design Decisions

Dict-based value representation: While SageLang recently added native C-like enums (Phase 1.7), the self-hosted interpreter relies on dictionary-based representations for its AST nodes because it was designed before these features existed and for simplicity. All AST nodes, functions, classes, and instances are represented as dicts with an `__interp_type` field:

```
# A function value
let fn = {}
fn["__interp_type"] = "function"
fn["name"] = "add"
fn["params"] = ["a", "b"]
fn["body"] = body_ast
```

Control flow signals: Return, break, and continue are implemented as dict values:

```
# return 42  →  {"kind": "return", "value": 42}
# break      →  {"kind": "break"}
# continue   →  {"kind": "continue"}
# normal     →  {"kind": "normal"}
```

GC must be disabled: The self-hosted interpreter creates many dict allocations, which can trigger GC segfaults. Always start with `gc_disable()`.

13.4 Feature Coverage

The self-hosted interpreter supports (~70% feature parity with C):

Fully implemented: bare `end` statements (optional block terminators), anonymous procedure expressions (`proc(params...): body [end]`), full CLI flag surface parity with C `main.c` (`-c`, `--jit`, `--aot`, `--sgvm`, `--lsp`), arithmetic, variables, control flow (if/elif/else, while, for), functions with closures, recursion, classes with inheritance, arrays, dicts, tuples, strings, slicing, indexing, try/catch/finally, break/continue, raise, module imports (`import X`, `import X as Y`, `from X import a, b`), property access/set, all bitwise operators (& | ^ ~ « »), `match`/`case`/`default` pattern matching, `defer` statements (LIFO cleanup on scope exit), generators/yield (eager collection via `next()`), 35+ builtins.

For self-hosted LLVM codegen specifically (`src/sage/llvm_backend.sage`), `from X import Y` constant imports are resolved during code generation for foldable top-level `let` values (including `as` aliases), matching the C LLVM backend behavior for cross-module constants.

Data Representation: The self-hosted interpreter relies on dictionary-based representations for its AST nodes, functions, classes, and instances, tagged with an `__interp_type` field, instead of native enums.

Delegated to host runtime: GC control (`gc_collect`, `gc_enable`, `gc_disable`, `gc_stats`), FFI (`ffi_open`, `ffi_close`, `ffi_call`, `ffi_sym`), memory access (`mem_alloc`, `mem_free`, `mem_read`, `mem_write`, `mem_size`, `addressof`), networking (via host `import` of native modules).

Stub/partial: async proc (registered with `is_async` flag, executes synchronously), await (evaluates expression directly).

Not implemented: true coroutine-based generators (uses eager collection instead), actual async threading in self-hosted path, GPU module (uses host `import gpu` directly).

Safety: while loop iteration limit (1M), recursion depth limit (50000 in the self-hosted interpreter, backed by the host's stack-proximity guard), rich error messages with source context via `errors.sage`.

13.5 Test Suite

The self-hosted tree includes core parser/interpreter suites plus additional tooling, optimizer, backend, compiler, error-reporting, LSP, and CLI tests. The foundational bootstrap suites are:

Category	Tests	Coverage
`test_lexer.sage`	13	Token types, indentation, keywords
`test_parser.sage`	130	All AST node types, operator precedence
`test_interpreter.sage`	18	Evaluation, scoping, closures, classes
`test_bootstrap.sage`	18	End-to-end: source → tokens → AST → result

The full test suite (interpreter + compiler + self-hosted tooling) totals **2060+ tests**.

Part 10: Native Standard Library Modules (Phase 11)

SageLang provides built-in native modules that are pre-loaded into the module cache and available via `import` without any file on disk.

10.1 Math Module

```
import math

print math.sqrt(16)    # 4
print math.sin(math.pi) # ~0
print math.floor(3.7)  # 3
print math.ceil(3.2)   # 4
print math.abs(-5)     # 5
print math.pow(2, 10)  # 1024
print math.log(math.e) # 1
math.printm("10 + 20") # Show work for 10 + 20
```

Available functions: `sqrt`, `sin`, `cos`, `tan`, `floor`, `ceil`, `abs`, `pow`, `log`, `printm` Constants: `pi` (3.14159…), `e` (2.71828…)

10.2 IO Module

```
import io

io.writefile("test.txt", "Hello, World!")
let content = io.readfile("test.txt")
print content  # Hello, World!

io.appendfile("test.txt", "\nLine 2")
print io.exists("test.txt")  # true

io.rename("test.txt", "renamed.txt")
io.remove("renamed.txt")

# Binary I/O
let buf = bytes("Hello\n")
io.writebytes("data.bin", buf)      # Write Bytes value (also accepts Array)
let r = io.readbytes("data.bin")    # Read as Bytes value
io.appendbytes("log.bin", buf)      # Append Bytes value
```

Available functions: `readfile`, `writefile`, `appendfile`, `exists`, `remove`, `rename`, `readbytes`, `writebytes`, `appendbytes`

Note: `io.readbytes` returns a `Bytes` value (byte buffer), and `io.writebytes`/`io.appendbytes` accept either `Bytes` or `Array` values.

10.3 String Module

```
import string

print string.char(65)        # A
print string.ord("A")        # 65
print string.startswith("hello", "he")  # true
print string.endswith("hello", "lo")    # true
print string.contains("hello", "ell")   # true
print string.repeat("ab", 3)            # abab
print string.reverse("hello")           # olleh
```

Available functions: `char`, `ord`, `startswith`, `endswith`, `contains`, `repeat`, `reverse`

10.4 Sys Module

```
import sys

print sys.platform()  # linux, darwin, or windows
print sys.version()   # Sage version string
let path = sys.env("HOME")
print path
```

Available functions: `args`, `exit`, `platform`, `version`, `env`, `setenv`

10.5 FAT Module

The native `fat` module provides early FAT filesystem parsing helpers for image inspection and kernel/boot tooling.

```
import os.fat
import io

let boot = io.readbytes("disk.img")
let info = fat.parse_boot_sector(boot)

print info["fat_type"]           # FAT8 / FAT12 / FAT16 / FAT32
print info["cluster_count"]
print fat.cluster_to_lba(info, 2)  # First data cluster -> LBA/sector index
```

Available functions: - `parse_boot_sector(bytes_array)` - `probe(path)` - `cluster_to_lba(info, cluster)` - `fat_entry_offset(info, cluster)`

Constants: - `FAT8`, `FAT12`, `FAT16`, `FAT32`

Current scope: - Boot sector parsing, FAT type detection, and layout math for FAT8/12/16/32. - Intended as a foundation for follow-up directory walking, FAT chain traversal, and read/write support.

Part 11: Concurrency and Async/Await (Phase 11)

11.1 Thread Module

The `thread` module provides low-level threading primitives backed by pthreads.

```
import thread

proc worker(name):
    print name
    return 42

# Spawn a thread
let t = thread.spawn(worker, "hello")

# Wait for result
let result = thread.join(t)
print result  # 42
```

Thread Functions

Function	Description
`thread.spawn(proc, args...)`	Spawn a new thread running `proc` with given arguments
`thread.join(t)`	Block until thread `t` completes, return its result
`thread.mutex()`	Create a new mutex
`thread.lock(m)`	Acquire mutex `m` (blocks if held)
`thread.unlock(m)`	Release mutex `m`
`thread.sleep(ms)`	Sleep for `ms` milliseconds
`thread.id()`	Return current thread identifier

Mutex Example

```
import thread

let m = thread.mutex()

proc critical_section():
    thread.lock(m)
    # ... protected code ...
    thread.unlock(m)
```

11.2 Async/Await

The `async proc` keyword declares a procedure that runs asynchronously. Calling it spawns a background thread and returns a thread handle. Use `await` to retrieve the result.

```
async proc compute(x):
    return x * x

# Calling an async proc spawns a thread
let future = compute(5)

# await blocks until the thread completes
let result = await future
print result  # 25
```

Parallel Execution

```
async proc slow_add(a, b):
    return a + b

# Launch two computations in parallel
let f1 = slow_add(1, 2)
let f2 = slow_add(3, 4)

# Await both results
let r1 = await f1
let r2 = await f2
print r1 + r2  # 10
```

How It Works

1. `async proc` is parsed as `STMT_ASYNC_PROC` and sets `is_async = 1` on the `FunctionValue`
2. When called, the interpreter pre-evaluates arguments and spawns a thread via `thread_spawn_native`
3. The call returns a `VAL_THREAD` value (a thread handle)
4. `await` on a `VAL_THREAD` calls `pthread_join` and returns the thread's result value

11.3 GC Thread Safety

The garbage collector is protected by a pthread mutex. All GC operations (allocation, collection, marking, sweeping) acquire the lock, ensuring safe concurrent allocation from multiple threads.

Part 12: Developer Tooling (Phase 12)

SageLang includes a complete set of developer tools for interactive development, code quality, and editor integration.

12.1 REPL (Read-Eval-Print Loop)

Launch the interactive REPL by running `sage` with no arguments or with the `--repl` flag:

```
sage
sage --repl
```

The REPL supports multi-line blocks (indented code is automatically continued), error recovery (errors are displayed without exiting), and built-in commands.

Built-in Commands:

Command	Description
`:help`	Show REPL usage information
`:quit` / `:exit`	Exit the REPL
`:vars [prefix]`	List current REPL bindings, optionally filtered by prefix
`:type <expr>`	Evaluate an expression and print its runtime type and value
`:load <file>`	Execute a Sage file inside the current REPL session
`:reset`	Reset the REPL environment and module cache
`:pwd`	Print the current working directory
`:cd <dir>`	Change the current working directory
`:cat <file>`	Print file contents. Uses `is_safe_path` (blocks single quotes, wraps in quotes).
`:ls [dir]`	List directory contents. Uses `is_safe_path`.
`:sh <cmd>`	Execute a shell command. Uses `is_safe_command` (blocks metacharacters like `;`, `&`, `|`, `>`).
`:gc`	Run garbage collection and print GC statistics
`:stats`	Show GC stats, stack depth, and CPU time

Note on REPL Security: The `:cat` and `:ls` commands validate arguments using `is_safe_path`, which allows spaces but blocks single quotes to prevent shell injection, wrapping the final argument in single quotes securely. The `:sh` command uses `is_safe_command` which allows both spaces and single quotes for arguments, but explicitly blocks shell metacharacters (`;`, `&`, `|`, `>`, etc.) for consistency with `sys.exec` hardening.

Example Session:

```
sage> let x = 10
sage> proc double(n):
....>     return n * 2
sage> print double(x)
20
sage> :quit
```

12.2 Code Formatter

The formatter normalizes indentation, spacing, and blank lines for consistent code style.

```
sage fmt program.sage             # Format file in place
sage fmt --check program.sage     # Check formatting (exit code 1 if changes needed)
```

Example:

```
$ sage fmt --check messy.sage
messy.sage: needs formatting
$ sage fmt messy.sage
Formatted: messy.sage
$ sage fmt --check messy.sage
messy.sage: already formatted
```

12.3 Linter

The linter performs static analysis with 13 rules across three categories:

```
sage lint program.sage
```

Rule Categories:

- `[W001]`: Unused variable warning.
- `[W002]`: Shadowed variable warning.
- `[W003]`: Unreachable code warning (e.g., after return/break/continue).
- `[W004]`: Empty block warning (e.g., following a colon).
- `[S003]`: Missing docstring warning (requires `##` preceding top-level procedures).
- `[S004]`: Trailing semicolon warning.
- `[S005]`: Multiple statements on a single line.

Category	Rules	Description
Errors	E001-E003	Syntax and structural errors
Warnings	W001-W005	Potential bugs and bad practices
Style	S001-S005	Code style and naming conventions

Notable Rules: - `[W001]`, `[W002]`: Issued for unused variables and variables that shadow existing declarations in scope. - `[W003]`: Warns about unreachable code following a `return`, `break`, or `continue` statement at the same indentation level. - `[W004]`: Warns about empty blocks (e.g., when a line ending in a colon `:` is followed by an empty or improperly indented section). - `[S003]`: Enforces documentation conventions; top-level `proc` declarations must be immediately preceded by a `##` style docstring (comment). Top-level procedure doc comments (`## ...`) must be placed before decorators like `@inline`. Placing them between `@inline` and `proc` triggers a parser error. - `[S004]`: Warns about trailing semicolons (not used in SageLang). - `[S005]`: Warns when multiple statements are on a single line separated by semicolons.

Example Output:

```
program.sage:5: W001 unused variable 'temp'
program.sage:12: E003 line too long (exceeds 120 characters)
program.sage:20: E001 Inconsistent indentation
```

12.4 Syntax Highlighting

SageLang provides editor support via TextMate grammars:

- **TextMate grammar**: `editors/sage.tmLanguage.json` works with any TextMate-compatible editor (VSCode, Sublime Text, etc.)

- **VSCode extension**: `editors/vscode/` provides full language support including syntax highlighting and language configuration

Installing the VSCode Extension:

1. Copy or symlink `editors/vscode/` into `~/.vscode/extensions/sage-lang/`
2. Restart VSCode
3. Open any `.sage` file to see syntax highlighting

12.5 Language Server Protocol (LSP)

The LSP server provides IDE-like features for any editor that supports the Language Server Protocol.

```
sage --lsp              # Start LSP server via main binary
sage-lsp                # Standalone LSP server binary
```

Capabilities:

Feature	Description
Diagnostics	Real-time error and warning reporting on file save
Completion	Keyword and symbol completions as you type
Hover	Type information and documentation on hover
Formatting	Format-on-save via `textDocument/formatting`

Editor Configuration (VSCode `settings.json`):

```
{
    "sage.lsp.path": "/path/to/sage-lsp"
}
```

Example Programs

Hello World:

```
print "Hello, World"
```

Factorial:

```
proc factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print factorial(5)  # 120
```

Fibonacci:

```
proc fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

for i in range(0, 10):
    print fib(i)
```

Object-Oriented:

```
class Rectangle:
    proc init(width, height):
        self.width = width
        self.height = height

    proc area():
        return self.width * self.height

let rect = Rectangle(5, 3)
print rect.area()  # 15
```

Part 14: Networking (Phase 14)

SageLang provides four native networking modules backed by libcurl and OpenSSL. These are implemented in C (`src/net.c`) and registered as native modules.

14.1 Socket Module

Low-level POSIX socket operations with constants for address families and socket types.

```
import socket

# TCP client
let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM, 0)
socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0" + chr(13) + chr(10) + chr(13) + chr(10))
let response = socket.recv(sock, 4096)
print response
socket.close(sock)

# DNS resolution
let ip = socket.resolve("example.com")
print ip  # "4.1.3.34"

# UDP
let udp = socket.create(socket.AF_INET, socket.SOCK_DGRAM, 0)
socket.sendto(udp, "hello", "4.1.3.1", 9999)
socket.close(udp)
```

Constants: `AF_INET`, `AF_INET6`, `SOCK_STREAM`, `SOCK_DGRAM`, `SOCK_RAW`, `IPPROTO_TCP`, `IPPROTO_UDP`

Functions (15): `create`, `bind`, `listen`, `accept`, `connect`, `send`, `recv`, `sendto`, `recvfrom`, `close`, `setopt`, `poll`, `resolve`, `getpeername`, `nonblock`

14.2 TCP Module

High-level TCP with automatic buffering and convenience functions.

```
import tcp

# Client
let conn = tcp.connect("example.com", 80)
tcp.sendall(conn, "GET / HTTP/1.0" + chr(13) + chr(10) + chr(13) + chr(10))
let line = tcp.recvline(conn)
print line
tcp.close(conn)
```

```
# Server
let server = tcp.listen("4.1.3.0", 8080, 5)
let client = tcp.accept(server)
tcp.send(client, "Hello from Sage!")
tcp.close(client)
tcp.close(server)
```

Functions (9): `connect`, `listen`, `accept`, `send`, `recv`, `sendall`, `recvall`, `recvline`, `close`

14.3 HTTP Module

HTTP/HTTPS client via libcurl. All request functions return a dict with `status`, `body`, and `headers` keys.

```
import http

# Simple GET
let resp = http.get("https://httpbin.org/get")
print resp["status"]   # 200
print resp["body"]

# POST with options
let opts = {"timeout": 30, "headers": {"Content-Type": "application/json"}}
let resp2 = http.post("https://httpbin.org/post", "{}", opts)

# Download file
http.download("https://example.com/file.txt", "/tmp/file.txt")

# URL encoding
print http.escape("hello world")    # "hello%20world"
print http.unescape("hello%20world") # "hello world"
```

Functions (9): `get`, `post`, `put`, `delete`, `patch`, `head`, `download`, `escape`, `unescape`

Options dict keys: `timeout` (seconds), `follow` (bool, follow redirects), `verify` (bool, SSL verification), `user_agent` (string), `headers` (dict of header name→value pairs), `cainfo` (CA certificate path)

14.4 SSL Module

OpenSSL TLS/SSL bindings for encrypted socket communication.

```
import socket
import ssl

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM, 0)
socket.connect(sock, "example.com", 443)

let ctx = ssl.context()
let s = ssl.wrap(ctx, sock)
ssl.connect(s)
ssl.send(s, "GET / HTTP/1.1" + chr(13) + chr(10) + "Host: example.com" + chr(13) + chr(10) + chr(13) + c
let data = ssl.recv(s, 4096)
print data

ssl.shutdown(s)
ssl.free(s)
ssl.free_context(ctx)
socket.close(sock)
```

Functions (13): `context`, `load_cert`, `wrap`, `connect`, `accept`, `send`, `recv`, `shutdown`, `free`, `free_context`, `error`, `peer_cert`, `set_verify`

14.5 Important Notes

- **No escape sequences**: Sage strings are raw. Use `chr(13) + chr(10)` for CRLF, `chr(34)` for double-quote, `chr(92)` for backslash.
- **HTTP module** handles HTTPS automatically via libcurl — no need for manual SSL setup for HTTP requests.
- **Build requirement**: libcurl and openssl development libraries must be installed.

Part 15: JSON Library (cJSON Port)

SageLang includes a complete 1:1 port of Dave Gamble's cJSON library in `lib/json.sage` (~1,050 lines). It uses a linked-list tree structure mirroring the C original.

15.1 Basic Usage

```
gc_disable()  # Required for class-heavy code

from json import cJSON_Parse, cJSON_Print, cJSON_PrintUnformatted
from json import cJSON_GetObjectItem, cJSON_GetStringValue, cJSON_GetNumberValue
from json import cJSON_ToSage, cJSON_Delete

# Parse JSON string (use chr(34) for double-quotes in Sage)
let json_str = "{" + chr(34) + "name" + chr(34) + ":" + chr(34) + "Alice" + chr(34) + "," + chr(34) + "a
let root = cJSON_Parse(json_str)

# Query values
let name = cJSON_GetStringValue(cJSON_GetObjectItem(root, "name"))
let age = cJSON_GetNumberValue(cJSON_GetObjectItem(root, "age"))
print name  # Alice
print age   # 30

# Print JSON
print cJSON_Print(root)           # Formatted (pretty-printed)
print cJSON_PrintUnformatted(root)  # Compact

# Convert to native Sage dict
let native = cJSON_ToSage(root)
print native["name"]  # Alice

cJSON_Delete(root)  # No-op in GC language, included for API compatibility
```

15.2 Creating JSON

```
gc_disable()
from json import cJSON_CreateObject, cJSON_CreateArray
from json import cJSON_AddStringToObject, cJSON_AddNumberToObject
from json import cJSON_AddBoolToObject, cJSON_AddNullToObject
from json import cJSON_AddArrayToObject, cJSON_AddItemToArray
from json import cJSON_CreateNumber, cJSON_PrintUnformatted

let obj = cJSON_CreateObject()
```

```
cJSON_AddStringToObject(obj, "name", "Bob")
cJSON_AddNumberToObject(obj, "age", 25)
cJSON_AddBoolToObject(obj, "active", true)
cJSON_AddNullToObject(obj, "deleted_at")

let tags = cJSON_AddArrayToObject(obj, "tags")
cJSON_AddItemToArray(tags, cJSON_CreateNumber(1))
cJSON_AddItemToArray(tags, cJSON_CreateNumber(2))

print cJSON_PrintUnformatted(obj)
# {"name":"Bob","age":25,"active":true,"deleted_at":null,"tags":[1,2]}
```

15.3 Sage-Native Conversion

```
gc_disable()
from json import cJSON_FromSage, cJSON_ToSage, cJSON_Print

# Native Sage value → cJSON tree
let data = {"users": [{"name": "Alice"}, {"name": "Bob"}], "count": 2}
let tree = cJSON_FromSage(data)
print cJSON_Print(tree)

# cJSON tree → native Sage value
let back = cJSON_ToSage(tree)
print back["users"][0]["name"]  # Alice
```

15.4 API Reference

Category	Functions
Parsing	`cJSON_Parse`, `cJSON_ParseWithLength`, `cJSON_GetErrorPtr`
Printing	`cJSON_Print`, `cJSON_PrintUnformatted`, `cJSON_PrintBuffered`
Creation	`cJSON_CreateNull/True/False/Bool/Number/String/Raw/Array/Obj` `CreateIntArray/DoubleArray/FloatArray/StringArray`
Query	`cJSON_GetArraySize`, `GetArrayItem`, `GetObjectItem`, `GetObjectItemCaseSensitive`, `HasObjectItem`, `GetStringValue`, `GetNumberValue`
Type Checks	`cJSON_IsInvalid/False/True/Bool/Null/Number/String/Array/Obj`
Array Ops	`AddItemToArray`, `InsertItemInArray`, `DetachItemFromArray`, `DeleteItemFromArray`, `ReplaceItemInArray`
Object Ops	`AddItemToObject/CS`, `DetachItemFromObject/CaseSensitive`, `DeleteItemFromObject/CaseSensitive`, `ReplaceItemInObject/CaseSensitive`
Helpers	`cJSON_AddNullToObject`, `AddTrue/False/Bool/Number/String/Raw/Array/ObjectToObject`
Utility	`cJSON_Duplicate`, `Compare`, `Minify`, `Delete`, `SetValuestring`, `SetNumberHelper`, `Version`
Sage Extras	`cJSON_ToSage` (tree→native), `cJSON_FromSage` (native→tree)

15.5 Important Notes

- **GC must be disabled** (`gc_disable()`) when using json.sage — class-heavy code triggers GC segfaults.
- `cJSON_GetObjectItem` does **case-insensitive** matching (uses `lower()`). Use `cJSON_GetObjectItemCaseSensitive` for exact matching.
- `cJSON_Delete` is a no-op (Sage has GC), included for API compatibility with C cJSON.
- **Type constants**: `cJSON_Invalid=0`, `cJSON_False=1`, `cJSON_True=2`, `cJSON_NULL=4`, `cJSON_Number=8`, `cJSON_String=16`, `cJSON_Array=32`, `cJSON_Object=64`, `cJSON_Raw=128`

Conclusion

SageLang is a comprehensive, well-structured scripting language for systems and embedded programming. Its design combines the approachability of Python with low-level control through direct memory access, FFI, and inline assembly. The phased development approach (Phases 1–18) progresses from core features through advanced topics including OOP, generators, compilation backends (C, LLVM IR, native assembly), concurrency, networking, JSON processing, and a self-hosted interpreter — all fully implemented and integrated.

Key Takeaways: - **Lexer + Parser + Interpreter** pipeline is modular and extensible. - **Value system** supports dynamic typing with heap-allocated complex objects and GC. - **Scoped environments** enable closures and lexical scoping. - **Exception handling**, **generators**, and **async/await** provide modern control flow. - **Module system** enables code reuse with native (C) and Sage library modules. - **Mark-and-sweep GC** manages memory automatically with thread safety. - **Multiple backends**: tree-walking interpreter, C codegen, LLVM IR, native assembly (x86-64, aarch64, rv64, mips). - **Networking**: POSIX sockets, TCP, HTTP/HTTPS (libcurl), SSL/TLS (OpenSSL). - **Self-hosted**: Lexer, parser, and interpreter ported to Sage with full bootstrap. - **Test suite**: interpreter/compiler coverage, JSON coverage, and broad self-hosted suites spanning parsing, tooling, optimization, codegen, compiler, LSP, and CLI behavior.

SageLang offers a practical balance between ease of use and systems-level control, making it ideal for prototyping, education, embedded scripting, and learning language implementation.

GPU Graphics (Vulkan + OpenGL)

SageLang includes a professional-grade graphics engine supporting both Vulkan and OpenGL 4.5 backends. The library is built in four layers:

1. **Pure C GPU API** (`gpu_api.h/gpu_api.c`) – Backend-agnostic ~100 functions, no interpreter dependency
2. **C native module** (`import gpu`) – Interpreter wrappers for Value-based argument handling
3. **Sage builder libraries** (`lib/vulkan.sage`, `lib/opengl.sage`, `lib/gpu.sage`) – Ergonomic helpers
4. **Engine + UI libraries** (`lib/renderer.sage`, `lib/pbr.sage`, `lib/ui.sage`, etc.) – Application-level systems

Three execution paths share the same GPU API:

- **Interpreter**: `graphics.c` wraps `sgpu_*` with Value extraction
- **LLVM compiled**: `llvm_runtime.c` provides 103 `sage_rt_gpu_*` bridge functions
- **Bytecode VM**: 30 dedicated `BC_OP_GPU_*` opcodes for frame-loop hot paths

Build Requirements

Vulkan and OpenGL support are auto-detected at build time via `pkg-config`. GLFW is required for windowed rendering:

```
# Auto-detect Vulkan + OpenGL (default)
make

# Force enable/disable
make VULKAN=1    # Force Vulkan
make VULKAN=0    # Disable Vulkan
make OPENGL=1    # Force OpenGL
make OPENGL=0    # Disable OpenGL
```

Without the Vulkan SDK, the `gpu` module loads in stub mode – all constants are available, functions return errors gracefully. Use `import graphics.opengl` instead of `import gpu` for the OpenGL 4.5 backend (same API, different initialization).

Quick Start: Empty Window

```
import gpu

gpu.init_windowed("My App", 800, 600, "Window Title", false)
print gpu.device_name()

let attach = {}
attach["format"] = gpu.FORMAT_SWAPCHAIN
attach["load_op"] = gpu.LOAD_CLEAR
attach["store_op"] = gpu.STORE_STORE
attach["initial_layout"] = gpu.LAYOUT_UNDEFINED
attach["final_layout"] = gpu.LAYOUT_PRESENT
let rp = gpu.create_render_pass([attach])
let framebuffers = gpu.create_swapchain_framebuffers(rp)

let cmd_pool = gpu.create_command_pool()
let cmd = gpu.create_command_buffer(cmd_pool)
let img_sem = gpu.create_semaphore()
let rdr_sem = gpu.create_semaphore()
let fence = gpu.create_fence(true)

while gpu.window_should_close() == false:
    gpu.poll_events()
    gpu.wait_fence(fence)
    gpu.reset_fence(fence)
    let idx = gpu.acquire_next_image(img_sem)
    gpu.begin_commands(cmd)
    gpu.cmd_begin_render_pass(cmd, rp, framebuffers[idx], [[0.1, 0.1, 0.2, 1.0]])
    gpu.cmd_end_render_pass(cmd)
    gpu.end_commands(cmd)
    gpu.submit_with_sync(cmd, img_sem, rdr_sem, fence)
    gpu.present(idx, rdr_sem)

gpu.device_wait_idle()
gpu.shutdown_windowed()
```

Quick Start: Triangle

```
import gpu

gpu.init_windowed("Triangle", 800, 600, "Sage Triangle", false)
```

```
# Load compiled SPIR-V shaders
let vert = gpu.load_shader("triangle.vert.spv", gpu.STAGE_VERTEX)
let frag = gpu.load_shader("triangle.frag.spv", gpu.STAGE_FRAGMENT)

# Render pass with swapchain format
let attach = {}
attach["format"] = gpu.FORMAT_SWAPCHAIN
attach["load_op"] = gpu.LOAD_CLEAR
attach["store_op"] = gpu.STORE_STORE
attach["initial_layout"] = gpu.LAYOUT_UNDEFINED
attach["final_layout"] = gpu.LAYOUT_PRESENT
let rp = gpu.create_render_pass([attach])

# Graphics pipeline
let layout = gpu.create_pipeline_layout([], 0)
let cfg = {}
cfg["layout"] = layout
cfg["render_pass"] = rp
cfg["vertex_shader"] = vert
cfg["fragment_shader"] = frag
cfg["topology"] = gpu.TOPO_TRIANGLE_LIST
cfg["cull_mode"] = gpu.CULL_NONE
let pipeline = gpu.create_graphics_pipeline(cfg)

# Render loop draws 3 hardcoded vertices
# gpu.cmd_bind_graphics_pipeline(cmd, pipeline)
# gpu.cmd_draw(cmd, 3, 1, 0, 0)
```

GPU Module API Reference

Context Lifecycle

Function	Description
`gpu.has_vulkan()`	Returns `true` if Vulkan is available
`gpu.has_window`	`true` if GLFW windowed mode is available
`gpu.initialize(name, validation?)`	Initialize headless Vulkan context
`gpu.init_windowed(name, w, h, title, validation?)`	Initialize windowed Vulkan + GLFW + swapchain
`gpu.shutdown()`	Destroy headless context
`gpu.shutdown_windowed()`	Destroy window + context
`gpu.device_name()`	GPU device name string
`gpu.device_limits()`	Dict of device limits

Windowing

Function	Description
`gpu.window_should_close()`	Check if window close requested
`gpu.poll_events()`	Process window events
`gpu.swapchain_extent()`	Dict `{width, height}`
`gpu.swapchain_image_count()`	Number of swapchain images
`gpu.acquire_next_image(semaphore)`	Get next image index (-1 on error)

Function	Description
`gpu.present(image_index, wait_semaphore)`	Present rendered image
`gpu.create_swapchain_framebuffers(render_pass)`	Array of framebuffer handles

Resources

Function	Description
`gpu.create_buffer(size, usage, memory)`	Create GPU buffer, returns handle
`gpu.buffer_upload(handle, float_array)`	Upload data to buffer
`gpu.buffer_download(handle)`	Download data from buffer
`gpu.create_image(w, h, d, format, usage)`	Create GPU image with auto view
`gpu.create_sampler(mag, min, address)`	Create texture sampler
`gpu.load_shader(path, stage)`	Load SPIR-V shader module

Descriptors

Function	Description
`gpu.create_descriptor_layout(bindings)`	Create from array of binding dicts
`gpu.create_descriptor_pool(max_sets, sizes)`	Create descriptor pool
`gpu.allocate_descriptor_set(pool, layout)`	Allocate a descriptor set
`gpu.update_descriptor(set, binding, type, resource)`	Bind buffer/image to descriptor
`gpu.update_descriptor_image(set, binding, image, sampler)`	Bind combined image sampler

Pipelines

Function	Description
`gpu.create_pipeline_layout(layouts, push_size?, stages?)`	Create pipeline layout
`gpu.create_compute_pipeline(layout, shader)`	Create compute pipeline
`gpu.create_graphics_pipeline(config_dict)`	Create graphics pipeline (see below)
`gpu.create_render_pass(attachments)`	Create render pass
`gpu.create_framebuffer(render_pass, images, w, h)`	Create framebuffer

Graphics pipeline config dict keys: - `layout`, `render_pass`, `vertex_shader`, `fragment_shader` (required) - `topology` (default: `TOPO_TRIANGLE_LIST`) - `cull_mode`, `front_face`, `polygon_mode` - `depth_test`, `depth_write` (booleans) - `blend` (boolean, enables alpha blending) - `vertex_bindings`, `vertex_attribs` (arrays of dicts)

Commands

Function	Description
`gpu.create_command_pool()`	Create command pool
`gpu.create_command_buffer(pool)`	Allocate command buffer
`gpu.begin_commands(cmd)`	Begin recording
`gpu.end_commands(cmd)`	End recording
`gpu.cmd_bind_compute_pipeline(cmd, pipe)`	Bind compute pipeline
`gpu.cmd_bind_graphics_pipeline(cmd, pipe)`	Bind graphics pipeline
`gpu.cmd_bind_descriptor_set(cmd, layout, index, set)`	Bind descriptor set
`gpu.cmd_dispatch(cmd, x, y, z)`	Dispatch compute workgroups
`gpu.cmd_push_constants(cmd, layout, stage, data)`	Push constants (float array)
`gpu.cmd_begin_render_pass(cmd, rp, fb, clear)`	Begin render pass
`gpu.cmd_end_render_pass(cmd)`	End render pass
`gpu.cmd_draw(cmd, verts, instances, first_v, first_i)`	Draw call
`gpu.cmd_draw_indexed(cmd, indices, instances, first, offset, first_i)`	Indexed draw
`gpu.cmd_bind_vertex_buffer(cmd, buffer)`	Bind vertex buffer
`gpu.cmd_set_viewport(cmd, x, y, w, h, min_d, max_d)`	Set viewport
`gpu.cmd_set_scissor(cmd, x, y, w, h)`	Set scissor rect
`gpu.cmd_pipeline_barrier(cmd, src, dst, src_acc, dst_acc)`	Memory barrier
`gpu.cmd_image_barrier(cmd, img, old, new, src, dst, s_acc, d_acc)`	Image barrier

Synchronization

Function	Description
`gpu.create_fence(signaled?)`	Create fence
`gpu.wait_fence(fence, timeout?)`	Wait for fence
`gpu.reset_fence(fence)`	Reset fence
`gpu.create_semaphore()`	Create semaphore
`gpu.submit(cmd, wait?, signal?, fence?)`	Submit to graphics queue
`gpu.submit_compute(cmd, wait?, signal?, fence?)`	Submit to compute queue
`gpu.submit_with_sync(cmd, wait_sem, signal_sem, fence)`	Full sync submit
`gpu.device_wait_idle()`	Wait for GPU idle

Key Constants

```
# Buffer usage (bitwise OR)
gpu.BUFFER_STORAGE  gpu.BUFFER_UNIFORM  gpu.BUFFER_VERTEX
```

```
gpu.BUFFER_INDEX    gpu.BUFFER_STAGING  gpu.BUFFER_TRANSFER_SRC

# Memory
gpu.MEMORY_DEVICE_LOCAL  gpu.MEMORY_HOST_VISIBLE  gpu.MEMORY_HOST_COHERENT

# Formats
gpu.FORMAT_RGBA8  gpu.FORMAT_RGBA16F  gpu.FORMAT_RGBA32F
gpu.FORMAT_R32F   gpu.FORMAT_DEPTH32F gpu.FORMAT_SWAPCHAIN

# Shader stages
gpu.STAGE_VERTEX  gpu.STAGE_FRAGMENT  gpu.STAGE_COMPUTE  gpu.STAGE_ALL

# Descriptor types
gpu.DESC_STORAGE_BUFFER  gpu.DESC_UNIFORM_BUFFER
gpu.DESC_SAMPLED_IMAGE   gpu.DESC_STORAGE_IMAGE  gpu.DESC_COMBINED_SAMPLER

# Topology
gpu.TOPO_TRIANGLE_LIST  gpu.TOPO_LINE_LIST  gpu.TOPO_POINT_LIST

# Pipeline stages (barriers)
gpu.PIPE_TOP  gpu.PIPE_COMPUTE  gpu.PIPE_TRANSFER  gpu.PIPE_FRAGMENT

# Access flags (barriers)
gpu.ACCESS_SHADER_READ  gpu.ACCESS_SHADER_WRITE
gpu.ACCESS_TRANSFER_READ  gpu.ACCESS_HOST_READ
```

Demos

Three demo programs are included in `examples/`:

Demo	File	Description
Empty Window	`examples/gpu_window.sage`	Creates a window with cycling clear color
Triangle	`examples/gpu_triangle.sage`	Classic colored triangle via vertex/fragment shaders
3D Hello World	`examples/gpu_hello3d.sage`	Rotating "HELLO WORLD" text rendered as 3D line segments

Run them with:

```
./sage examples/gpu_window.sage
./sage examples/gpu_triangle.sage
./sage examples/gpu_hello3d.sage
```

Shaders are pre-compiled SPIR-V files in `examples/shaders/`. Recompile with:

```
cd examples/shaders
glslc triangle.vert -o triangle.vert.spv
glslc triangle.frag -o triangle.frag.spv
glslc text3d.vert -o text3d.vert.spv
glslc text3d.frag -o text3d.frag.spv
```

9.21 Hardware Natives (Embedded Targets)

For embedded targets (such as the Pico or other bare-metal environments), the SageLang C backend provides native hardware interaction routines through the `hw` namespace. These bindings map directly to standard C

hardware APIs.

Function Signature	Description
`hw.gpio_init(pin)`	Initializes a GPIO pin.
`hw.gpio_set_dir(pin, out)`	Sets the direction of a GPIO pin (e.g., input or output).
`hw.gpio_put(pin, val)`	Sets the output value of a GPIO pin.
`hw.gpio_get(pin)`	Reads the value of a GPIO pin.
`hw.gpio_set_pull(pin, up, down)`	Configures pull-up/pull-down resistors for a pin.
`hw.clock_hz()`	Returns the system clock frequency in Hz.
`hw.uptime_ms()`	Returns the system uptime in milliseconds.
`hw.delay_ms(ms)`	Blocks execution for `ms` milliseconds.
`hw.delay_us(us)`	Blocks execution for `us` microseconds.
`hw.uart_init(baud)`	Initializes the default UART interface at the specified baud rate.
`hw.uart_putc(c)`	Transmits a single character over UART.
`hw.uart_puts(s)`	Transmits a string over UART.
`hw.uart_getc()`	Reads a single character from UART (returns -1 if no data is available).
`hw.adc_init(pin)`	Initializes an ADC (Analog-to-Digital Converter) pin.
`hw.adc_read(pin)`	Reads the analog value from the specified ADC pin.
`hw.temp_c()`	Returns the internal temperature sensor reading in degrees Celsius.
`hw.rgb_set(r, g, b)`	Sets the color of an onboard RGB LED (e.g., WS2812).
`hw.spi_init(bus, baud)`	Initializes an SPI bus.
`hw.spi_write(bus, data)`	Writes data to the SPI bus.
`hw.spi_read(bus, count)`	Reads `count` bytes from the SPI bus.
`hw.lcd_fb_init(w, h)`	Initializes an LCD framebuffer of width `w` and height `h`.
`hw.lcd_fb_pixel(x, y, color)`	Sets the color of a specific pixel in the LCD framebuffer.
`hw.lcd_fb_fill(color)`	Fills the entire LCD framebuffer with `color`.
`hw.lcd_fb_flush_bytes(count)`	Flushes `count` bytes of the framebuffer to the LCD screen.

Note: These functions are resolved during compilation by the C backend when targeting appropriate platforms.

Appendix: Quick Reference

Keywords

```
and as async await break case catch class comptime continue default defer elif else end enum false final
```

Note: `print`, `end`, `match`, `init`, `enum`, `struct`, and `trait` are soft keywords, meaning they can be used as variable, property, or method names where unambiguous.

Built-in Functions

```
len(x) push(arr, val) pop(arr) append(arr, val) range(a, b)
split(str, delim) join(arr, sep) replace(s, old, new)
upper(s) lower(s) strip(s) slice(arr/str, start, end)
str(x) tonumber(s) int(x) input() clock()
type(x) val_tag(x) build_info() chr(n) ord(c) hash(x) sizeof(x) doc(x)
startswith(s, prefix) endswith(s, suffix)
contains(s, sub) indexof(s, sub) string_count(s, sub) string_repeat(s, n)
array_extend(a, b) array_repeat(a, n) array_reverse(a)
array_contains(a, v) array_index_of(a, v) array_min(a) array_max(a)
array_sum(a) array_product(a)
bytes() bytes_len(b) bytes_get(b, i) bytes_set(b, i, v)
bytes_push(b, v) bytes_slice(b, s, e) bytes_to_string(b)
dict_keys(d) dict_values(d) dict_has(d, k) dict_delete(d, k) hasattr(obj, name)
gc_collect() gc_stats() gc_collections() gc_enable() gc_disable()
gc_mode() gc_set_arc() gc_set_orc()
next(gen)
ffi_open(path) ffi_call(lib, fn, ret, ...) ffi_sym(lib, name) ffi_close(lib)
ffi_sym_addr(lib, name)
mem_alloc(n) mem_free(ptr) mem_read(ptr, off, type) mem_write(ptr, off, type, val)
mem_size(ptr) addressof(val) addressof_raw(val) ptr_add(ptr, n) ptr_to_int(ptr)
path_join(a, b) path_dirname(p) path_basename(p) path_ext(p)
path_exists(p) path_is_dir(p) path_is_file(p)
asm_exec(code, ret, ...) asm_compile(code, arch, out) asm_arch()
struct_def(fields) struct_new(def) struct_get(ptr, def, name)
struct_set(ptr, def, name, val) struct_size(def)
cpu_count() cpu_physical_cores() cpu_has_hyperthreading()
thread_set_affinity(id) thread_get_core()
atomic_new(v) atomic_load(a) atomic_store(a, v)
atomic_add(a, v) atomic_cas(a, e, d) atomic_exchange(a, v)
sem_new(n) sem_wait(s) sem_post(s) sem_trywait(s)
vm_gas_limit_set(n) vm_gas_used_get() vm_gas_limit_get()
build_quad_verts(quads) build_line_quads(lines, thickness, r, g, b, a)
```

Operators

```
Arithmetic:    + - * / %
Comparison:    == != < > <= >=
Logical:       and or not
Bitwise:       & | ^ ~ << >>
Assignment:    =
Indexing:      arr[i]
Slicing:       arr[a:b]
Property:      obj.field  or  obj->field
Call:          func(args)
```